

Clifford Algebra Calibration Post-Quantum Cryptography

G. P. Wilmot, J. Chappell, D. K. Abbott

Abstract

The double lock scenario allows a secret to be transferred from one person to another without having previously exchanged secrets such as private/public keys. This can be realised by rotations of generalised calibrations in Clifford algebra. Calibrations map to Cayley-Dickson algebras and generalising this provides the mathematical structure enabling the enormous code space needed for a cryptosystem. Commuting rotations in large dimensional spaces are used but the complexity comes from the message being embedded in generalised calibrations. This system uses ideals of Clifford algebras and quasi-algebras that are sometimes embedded in Cayley-Dickson algebras making it a candidate for post-quantum cryptography. Understanding whether the new mathematics of generalised calibrations can produce a secure cryptosystem is the challenge presented in this manuscript.

Clifford algebras, quasi-Cayley-Dickson algebras, calibrations, post-quantum cryptography

1 Introduction

The double lock box is an example of symmetric encryption whereby a secret is passed between two agents without prior knowledge being exchanged. It was theorised by Kish-Sethuraman [1] (KS) but was already superseded by the Diffie-Hellman system, which requires fewer transmissions to establish a secure channel. In fact, the factorisation of large primes used in the original implementation of Diffie-Hellman can be used in the KS system [2] with less efficiency. But the double lock box admits a solution that is akin to information-set decoding, so can be considered for post-quantum cryptography. The KS cipher, as described by Chappell [3], and shown

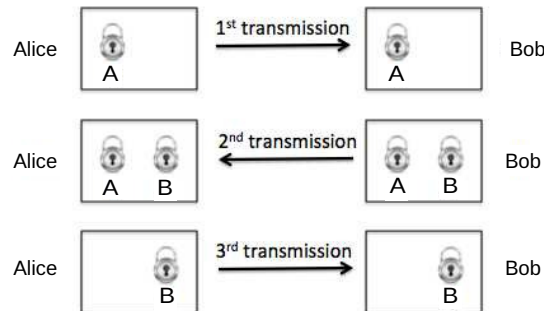


Figure 1: Transmission scheme

in Figure 1, involves Alice constructing a secure, tamper evidence box with double

latches. A secret is placed inside and a padlock on the outside. This is posted to Bob who adds another padlock in parallel to the first padlock and returns the box. Alice then unlocks the first lock and again forwards the box to Bob who unlocks the second lock to retrieve the secret. The digital version of this scheme would transmit the digital box in the open, via email for example. To enable such a scheme, KS show that the lock operators, $\mathcal{A}$ and $\mathcal{B}$, must commute. If the message in the box is represented as m , then

$$\mathcal{A}(\mathcal{B}m) = \mathcal{B}(\mathcal{A}m). \quad (1)$$

The construct used here is orthogonal rotations so that $\mathcal{A}\mathcal{B} = \mathcal{B}\mathcal{A}$, implying no intersection of orthogonal dimensions between $\mathcal{A}$ and $\mathcal{B}$.

The orthogonal rotations for $\text{Cl}(n)$ use the Spin group with basis e_{ij} such that $\phi' = \theta_{ij}(\phi) = R_{ij}\phi R_{ij}^{-1}$, where $R_{ij} = \cos(\frac{\pi}{4}) + e_{ij} \sin(\frac{\pi}{4})$, $i \neq j \in \mathbb{N}_1^n$. The rotated calibration ϕ' has the form of ϕ but with different signs and 3-forms but the inverse rotation, θ_{ji} will return ϕ . Now since orthogonal rotations commute $[e_{ij}, e_{kl}] = 0$, for $k, l \neq i, j$, then

$$\theta_{lk}(\theta_{ji}(\theta_{kl}(\theta_{ij}(\phi)))) = \phi.$$

This is a symmetric encryption scheme where Alice has used rotation θ_{ij} and Bob has used θ_{kl} without prior knowledge other than agreement of orthogonality. The size can be inferred from the message size. For the very small example of seven bits there are only 3840 possible outcomes and less than 21 total rotations, which are the keys to the locks in this encryption scheme.

Equation (1) is usually interpreted with m being a large integer or a vector so that $\mathcal{A}m$ has the same form or properties. But Clifford algebra offers another message format that has properties whereby $\mathcal{A}m$ is not a linear combination of m . These are generalised calibrations that are akin to using information-set codes for each bit of the message. The rotations not only change the codes but also the values of the bits. The number of codes increases as $O(n)$ over the number of rotations, $\binom{n}{2}$, which has $O(n^2)$ complexity increase.

Calibrations in the positive definite Clifford algebra, $\text{Cl}(n)$, $n = 2^N - 1$, $N > 1$ are shown to map to all Cayley-Dickson algebras [5]. Calibrations specify faces and parity of generalised polygons or polytopes with n completely connected vertices by arrows called a directed $(n - 1)$ -simplex. Calibrations represent coverage of all edges of the simplex by faces that only intersect at vertices, which is designated independent faces because their parities are independent. This means any pair of dimensions are uniquely associated with another dimension, not unique for $n > 3$, which defines a cross product. This, in turn, defines a multiplication table and hence an algebra. Some of these generalised calibrations of $\text{Cl}(n)$ map to Cayley-Dickson algebras but others are shown to map to quasi-Cayley-Dickson algebras [4, 5]. This mapping is only important in the context of the cryptosystem in order to prove that the generalised calibrations are well defined.

Definition. The generalised calibrations are divided into calibrations that map to Cayley-Dickson algebras and quasi-calibrations that map to quasi-Cayley-Dickson algebras. Reference [5] showed that for $N = 3$, calibrations transform to other calibrations under any rotation and hence, since rotations are invertible, quasi-calibrations transform amongst themselves. Calibrations are a much smaller subset of generalised calibrations and this needs to be taken into consideration later.

Definition. A quasi-Cayley-Dickson algebra is identified by the $\mathbb{P}_k$ code where k is the number of non-associative unique triads, $[a, b, c] \neq 0$, for $a < b < c$, and the

product $[a, b, c] = (ab)c - a(bc)$ is defined by the terms of the calibration $\pm e_{ijk}$ s.t. if $a = e_i$ and $b = e_j$ then $ab = \pm e_k$ and $e_j e_i = \mp e_k$. The number of quasi-Cayley-Dickson algebras is six for $Cl(7)$ and 100 for $Cl(15)$ and is unknown thereafter but the number of primary representations of calibrations is called the *representation space*, which is an extremely large space that enables the security of the cryptosystem.

Table 1: Generalised Calibration Dimensions

Level	Dimension	Rotations	Msg Bit Length	Rep. Space
3	7	21	7	30
4	15	105	35	64.8×10^6
5	31	465	155	8.2×10^{26}
6	63	1953	651	9.8×10^{76}
7	127	8001	2667	1.8×10^{199}
8	255	32,385	10,795	6.1×10^{485}
9	511	130,305	43,435	$1.9 \times 10^{1,140}$
10	1,023	522,753	174,251	$5.3 \times 10^{2,607}$

The dimensions, number of rotations, sizes of the generalised calibration and representation space sizes are provided in Table 1 for levels up to $N = 10$. The message length is the number of codes or terms in the generalised calibration. The number of codes available as 3-forms, the $\mathbb{P}_k$ code and other parameters such as the number of calibrations and quasi-calibrations are provided in Table 2.

An example provides an ideal way to introduce this encryption system. For $N = 3$ with seven dimensions the first generalised calibration from [4] is

$$\phi_1 = e_{123} + e_{145} + e_{167} + e_{246} + e_{257} + e_{347} + e_{356}.$$

This is a 3-form with seven terms called a primary since it has all positive terms. Any set of sign variations can be selected to represent bits with $+$ $\rightarrow$ 1 and $-$ $\rightarrow$ 0.

The set of $\binom{7}{2} = 21$ 90° basis rotations or 2-forms in $Cl(7)$, applied repeatedly, was shown in [4] to generate 30 other 3-forms that cover all the $\binom{7}{3} = 35$ single 3-forms in $Cl(7)$. Changing the signs of certain terms provides a calibration that maps to octonions, $\mathbb{O}$, and reflections generate another 15 maps to $\mathbb{O}$ thus generating the 480 representations of octonions. The remaining $30 \times 2^7 - 480$ sign variations generate six other algebras that are power-associative and called quasi-octonions and these have many representations. This is the mathematical underpinning of the calibration cryptographic technique. The 3-form terms act as information-set codes with $30 \times 112 = 2240$ possible outcomes if a quasi-calibration was the starting point. The rotations can be performed in any order and will only commute if orthogonal. Octonions are the last of the normed Cayley-Dickson algebras with higher levels being power-associative containing zero-divisors. The sedenions, $\mathbb{S}$, at level $N = 4$, have 15 vertices of the 14-simplex represented by the 15 basis elements of $Cl(15)$. The possible outcomes here are $32528 \times 64.8 \times 10^6$.

The octonion example here is minimal in that only seven bits can be encoded. But Table 1 demonstrates the message length and number of rotations increase quickly. Given that it is the permutations of the rotations with up to $\lfloor \binom{n}{2}/2 \pm 1 \rfloor!$ possibilities for Alice and Bob then the complexity has increased exponentially as seen by the representation space column of Table 1. This column is the number of primary calibrations available, one of which has the signs in the correct order to be the cipher.

Table 2: Generalised Calibration Parameters

Level	Codes	$\mathbb{P}_k$ Code	Internal Cal. Count	All-Even
3	35	28	127	1
4	455	252	32768	7
5	4495	2156	2.1×10^9	35
6	39,711	18,732	9.2×10^{18}	155
7	333,375	159,852	1.7×10^{38}	651
8	2,731,135	1,331,820	5.8×10^{76}	2667
9	22,108,415	10,902,892	6.7×10^{153}	10,795
10	177,910,271	88,310,124	9.0×10^{307}	43,435

The columns in Table 2 are the number of Codes, which is the number of distinct 3-forms used in the Message Length of Table 1, the $\mathbb{P}_k$ Code for the calibration and the number of generalised calibrations within a primary, which is the number of permutations of signs in each primary calibration, 2^m where m is the message bit length. This totals to the number of calibration and quasi-calibrations. Note that $\mathbb{P}_{28}$ is used to represent the calibration that maps to octonions, even though octonions are not power associative. All other $\mathbb{P}_k$ codes map to power associative algebras. As stated above, there are six quasi-octonions and we later find 100 quasi-sedenions alongside sedenions with code $\mathbb{P}_{252}$. It is easy to see the Calibration Count is much smaller than the number of quasi-calibrations, which is analysed further because the octonion calibrations are invariant under all Pin transformations, which limits their applicability as a cryptosystem.

The mathematics that underpins this cryptosystem is derived from ideals of $\text{Cl}(n)$ subalgebras, called Sharp(n) algebras. These algebras are new and specify the structure of Cayley-Dickson algebras by identifying non-associative rings related to the associative generalised calibrations. This is described in the following section. Section 3 describes the encryption/decryption code presented in Appendix A. Section 4 provides mathematical tests of the code by verifying some of the numbers in Table 2. This code is presented in Appendix B and includes finding the 100 quasi-sedenion calibrations provided in Appendix C.

2 Mathematical Theory

Clifford algebras are represented as directed simplices and can be visualised as projections for sizes up to $N \leq 4$. For the dimensions selected here $n = 2^N - 1$, $N \geq 2$, these is the same infinite series as the finite geometry group $\text{PG}(N, 2)$ [5]. For example, the projection of the 6-simplex from a point at infinity to a plane gives the Fano plane, $\text{PG}(2, 2)$, shown in Figure 2.

For the selected dimensions of $\text{Cl}(n)$, $n = 2^N - 1$, $N > 1$, then the number of basis rotations is $\binom{n}{2}$, as shown in Table 1. This is also the number of edges in the $(n-1)$ -simplex and is always divisible by three since $\binom{n}{2}$ is a product of two consecutive numbers less than a power of two and this gives the number of faces that cover each edge once. This property is the main characteristic that defines calibration structures. The number of faces of this simplex is $\binom{n}{3}$ and a third of these can be selected that are independent and cover all edges so the primary calibration consists $\frac{1}{3}\binom{n}{2}$ 3-forms and this is provided in the Message Bit Length column of Table 1.

The $\mathbb{P}_k$ column where k is the code is derived from the number of subalgebras found in [5] and copied in Table 3. The value of k for each level is the sum of each

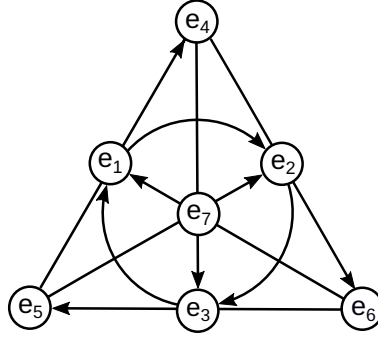


Figure 2: Fano plane calibration diagram

subalgebra's count multiplied by its k value where $k = 28$ for $\mathbb{O}$.

Table 3: Subalgebra Counts

Level	3	4	5	6	7	8	9	10
$\mathbb{O}$	1	8	50	310	2046	14,478	107,950	831,470
$\mathbb{P}_4$	0	7	63	413	2583	16,905	118,251	873,901
$\mathbb{P}_{12}$	0	0	42	504	4158	30,996	229,194	1,729,728
$\mathbb{P}_{14}$	0	0	0	168	3024	34,776	332,640	2,912,616

The generalised calibration count within each primary is just the permutations of all the signs, which is $2^n - 1$ where $n = 2^N - 1$. These are called internal generalised calibrations and are shown in Table 2. This number multiplied by the representation space size from Table 1 provides the number of possible outputs for the encoded message. The size of the calibration representation space is given by the following.

Theorem. For $P(n)$ being the number of primary calibration representations at level n then the recursive formula is

$$P(n) = \frac{(n-1)!}{(\frac{1}{2}(n+1))!} P(n-1), \quad \text{for } n = 2^N - 1, N \in \mathbb{N}_2^\infty. \quad (2)$$

Proof. Reference [5] showed a relationship between the sedenion associative calibration and the non-associative calibration. We assume here that this correspondence occurs for all higher levels, $N > 4$, which allows the later to define the automorphisms of the former. Since both calibrations have the same representation space of 64.8 million generalised calibrations, as proved by the test code in Appendix B, then this seems highly likely. An associative calibration has $\binom{n}{2}/3$ 3-form terms that represent Cayley-Dickson algebras consisting of this many quaternions. The non-associative calibration consisting of n terms is invertible so has a dual by multiplying it with the $\text{Cl}(n)$ pseudoscalar to bring it into the Sharp algebra as an $((n+1)/2)$ -form. Since this has even degree it generates 90° rotations that are automorphisms and define 2-form automorphisms of the Cayley-Dickson algebra. Each of the n terms thus provides $((n+1)/2)!$ permutations that need to be replaced with the permutations provided by the subalgebras, which are the representations of the calibrations given by each term and have sizes $P(n-1)$. Hence the total number of permutations, $n!$, is divided by the permutations of the calibrations of the subalgebras, $n((n+1)/2)!/P(n-1)$. $\square$

Generalised calibrations can be generated manually by building upon the previous level. We start with $N = 2$, giving dimension $n = 3$, and the calibration is

e_{123} , which designates the $\text{Cl}(3)$ quaternions, e_{12} , e_{23} and e_{13} . The opposite parity, $-e_{123}$, maps to anti-quaternions where the product is $+1$. The next level is $N = 3$ with dimension $n = 7$ and to build upon e_{123} we add 3-forms that contain pairs of indices not already used, with the lowest indices included first. Using the indices 4, 5, 6 and 7 builds the seven terms of ϕ above. The algorithm to generate primary calibrations is simply sum all simplex faces $\sum_{\mu \in \binom{n}{3}} e_{\mu_1 \mu_2 \mu_3}$, if any ordered pair, (μ_1, μ_2) , (μ_2, μ_3) , (μ_1, μ_3) , is not already included.

This generates the first primary with independent faces that cover all edges once and applying a basis $\text{Spin}(n)$ rotation to this will generate another structure with the same property. Taking the absolute value and selecting unique structures allows this process to be repeated for all basis 2-forms until no new structures are found. Such a process is used in Section 4 to generate all 64.8 million primaries that have associative calibrations that generate $\mathbb{S}$. The same code is used to generate 64.8 million 7-form non-associative calibrations, indicating that the assumption in the theorem that the two types of calibrations are related is correct.

The rotations for Alice and Bob must be orthogonal and a fixed scheme could be shared as prior knowledge or as part of a standard. Choosing small and large dimensions for rotations will favour left and right parts of the calibration so even and odd dimensions are the obvious choice. This brings the number of potential rotations to slightly less than half for all levels. For $N = 4$ there are 28 all-odd rotations and 21 all-even rotations giving a ratio of $49/105 = 0.47$ available reflections and this ratio increases as the level increases. Even though this limits the rotations to about 24% of the available rotations for each agent, these rotations can be repeated and this is needed to generate the large representation space for level $N = 4$ in the test code of Appendix B. It is also advantageous providing more opportunity for non-commuting rotations to occur.

One of the authors has pointed out that the octonion calibration contains an all-even 3-form, e_{246} , so initial even rotations would not encrypt this 3-form. The All-Even column of Table 2 shows that the number of all-even 3-forms increases as the level increases. There are no all-odd 3-forms in the first calibrations generated up to level $N = 10$ so the sender of the messages, Alice, must choose all odd rotations while Bob chooses all even. Some of Alice's encrypted bits would then not be changed by Bob, making cracking the full code slightly easier. As more rotations are performed then all-odd cases increase and all-even 3-forms decrease evening out this relationship so that on average about 15% of 3-forms will not be changed by Bob. Worse than this, after Alice un-rotates the third message, the all-even 3-forms will be in the clear or unencrypted. Both circumstances can be avoided if Alice avoids the all-even 3-forms in the original message by spreading the message or placing the bits at these location at the end of the message. Bob will need to take note of these all-even 3-forms in the first transmitted message and apply the same scheme. Hence the efficiency of the encryption scheme has decreased from 25% to 21% but increasing the number of repeats of the rotations will compensate for this.

The other special case mentioned in the introduction is that if the encoded bits generate a calibration then the output within the representation space is diminished. This is the case for octonions but for sedenions it is found that rotations within Fano planes and between $\mathbb{O}$ planes or between $\mathbb{P}_4$ planes has this property. But basis $\text{Spin}(15)$ rotations that mix $\mathbb{O}$ and $\mathbb{P}_4$ Fano planes change the sedenion calibration into a quasi-calibration. Of the 105 basis rotations 49 have this property, covering all-even and all-odd rotations, which means the chance of starting with a calibration and not rotating to a quasi-calibration is very small. The test code mentioned

below needs to rotate calibrations many times in order to find all possible 64.8 million primaries so an alternative scheme to increasing the level would be to repeat the random assignment of all possible rotations a few times to increase complexity without much extra work.

The test code uses ASCII characters ending in a control-d and the remainder of the message length is left un-randomised. Having a better termination scheme and randomising the remaining bits should be considered. Such schemes and those to avoid all-even 3-form triads are not included in the code provided. It is also imagined that ASCII is not the ideal encoding.

3 Cryptosystem Description

The Python code is presented in Appendix A. To create an encrypted message, run the following command line for Windows (for Linux replace py with python or python3),

```
py CalCrypt.py N <base filename> <cipher text>
```

Thereafter, the output file is transmitted to the other agent and the saved file processed as

```
py CalCrypt.py <saved filename>
```

On the first usage, the text provided on the command line is packed into single 7-bit bytes, rotated a random number of times with odd dimension rotations and added to the calibration structure, ready for transmission. After transmission, each receiver must unpack the transmission into the <saved filename> for further processing.

The second usage rotates the calibration structure with random even dimension rotations. Both sets of rotations are stored and used in reverse to un-rotate the calibration structure on subsequent processing stages. On the third usage of this command, the decoded cipher is written to the output file. This sequence can be run on the same machine for investigation purposes.

The level $N = 8$ with the 255 dimensions, represented by a single byte is likely to provide a practical encryption scheme. It has a code length of 10,795 bits, a transmission length of 33KB including 1 bit for each sign and stored agent rotation keys selected from about a quarter of the $\binom{255}{2} = 32,385$ possible rotations. Starting with any calibration structure then the secret is encoded in the 10,795 signs of the 3-form structure.

The timings for an 13th Gen Intel(R) Core(TM) i9-13900KF are 2.3 seconds to create the calibrations and 25.8 seconds to perform Alice's rotations. The later timing will be the same for all subsequence steps. The calibration time could be shortened using multiple threads.

4 Test Code Description

The Python code is presented in Appendix B and has only one command line,

```
py CalTest.py [options] N
```

Use the“-” option to view the list of options available.

The case $N = 3$ will generate a list of the 30 primary associative calibrations in 7 dimensions in string format in one file and a structure specifying the six quasi-octonion codes and first calibration in a separate file. The format of this second file is:

```
[ ([Pcode, ...], [Pindex, ...], Uindex, primary)], (...) ]
```

where Pcode is the $\mathbb{P}_k$ code, Pindex is the signed offset for this Pcode within the primary, Uindex is the signed offset for the first calibration and the primary is represented as string format, which are triples of hexadecimal digits for each 3-form. The Pcodes are unique and represent the first primary calibration and the first occurrence at the beginning of this primary is provided by the respective Pindex. This takes less than a second to run.

The case $N = 4$ generates the 64.8 million primary associative calibrations in 15 dimensions and the 100 quasi-sedenion quasi-calibrations. The output files are in batches of 2 million calibrations in order for Windows platforms to be able read the files. The files have the same format as above but takes a long time to run, as discussed below.

The case $N = 7$ generates the 64.8 million non-associative calibrations for 15 dimensions. This again writes successive files with 2 million calibrations each with 105 hexadecimal characters per line but these are interpreted as 15 lots of 7 digits for each 7-form. There are no quasi-calibrations generated with this option.

The two 15-D cases take many days to run on the HPC Phoenix system at Adelaide University. Since this system has a time limit of 3 days then options are available to process both the calibration generation phase and quasi-calibration phase in batches. The Processed Size reported in a previous run is used in the “-l” option and a time of one hour less than 3 days is used in the “-t” option to stop processing and write the results to the output files. Using the “-p4” option reads these files before processing starts. The initial run uses a Processed Size value of zero. The following slurm command is used to run the code on the Phoenix HPC.

```
sbatch -p icelake --mem 80GB -n 70 --time 2:59:59 \
--export=ALL,LEVEL=4,TIME=71,LAST=<Processed Size> calCrypt.sh
```

The calCrypt.sh bash script has the following contents.

```
./CalTest.py -vc $((($SLURM_NPROCS -1)) -t $TIME -l $LAST \
-p $((($LAST==0) ?3 :4)) -d "/hpcfs/users/<user>" $LEVEL
```

The process is repeated until the Process Size no longer increases. The processing results for the 100 quasi-calibrations at level $N = 4$ are presented in Appendix C. These results were originally generated using a Clifford algebra calculator and was able to generate all 15-D associative calibrations within 3 days using 80 cores and the 100 quasi-calibrations in another 2.5 days. The difference in time is that the calculator is optimised for 15-D while the cryptocode is unlimited up to practical time and memory resource limits. Hence this code is presented as a test of the system’s ability to generate valid quasi-calibrations at known levels.

The Clifford/Cayley-Dickson algebras calculator is also written in Python and this was then translated to the standalone CalCrypt and CalTest packages. The github URL for the calculator and the code presented in Appendices A and B is

<https://github.com/GPWilmot/geoalg>.

5 Summary

This cyptosystem has the ingredients to provide a post-quantum cryptosystem. The only prior knowledge required is the encoding of the binary string being transferred. The code presented here used ASCII for easier demonstration but any encoding scheme could be included in the standard. This paper presents this cryptosystem for the cryptography community to analyse in order to evaluate its potential.

The authors acknowledges the support of an Australian Government Research Training Program Scholarship. Support from the Australian Research Council (FL240100217) is also gratefully acknowledged.

References

- [1] L. B. Kish, S. Sethuraman, P. Heszler, Non Breakable Data Encryption With Classical Information? AIP Conf. Proc. **800** p.p. 193–196, 2005.
- [2] A. Klappenecker, Remark on a “Non-Breakable Data Encryption” Scheme by Kish and Sethuraman, Fluctuation and Noise Letters, **4:25**, 2004.
- [3] J. M.Chappel, L, J.Gunn, D. Abbott, Hot Topics in Physical Information (HoTPI-2013), International Journal of Modern Physics: Conference Series **33** 1460355, 2014.
- [4] G. P. Wilmot, Construction of Exceptional Lie Algebra G2 and Non-associative Algebras Using Clifford Algebra, Adv. Appl. Clifford Algebras, **36**, 24 (2026). <https://doi.org/10.1007/s00006-025-01423-5>.
- [5] G. P. Wilmot, Sedenion Automorphisms, arXiv:2512.07210 [math.RA], 2026.

Appendix A - Cryptosystem Code

```
#!/usr/bin/env python3
#####
## File: CalCrypt.py - symmetric encryption - G.P.Wilmot (c) Mar 2026
#####
import sys, random, time

class CalCrypt():
    """Class to create quasi-calibration at level N=3..10 and encrypt
    with setCode() and rotate() or decrypt using reverse() first.
    Calibrations map to Cayley-Dickson algebras and the Sharp(N)
    algebras in Cl(n), n=2*N-1, calculate them for all N>1."""

    def __init__(self, N, harpCal=None):
        """Create the initial calibration structure for this level."""
        self.__checkType(N, int, "init", (3,10))
        n = int(pow(2, N)) - 1
        if harpCal is None: # Build calibration
            got = set([])
            cal = []
            for tri in self.__comb(n, 3, list(range(1, n + 1))):
                pair1 = tuple(tri[0:2])
                pair2 = tuple(tri[1:3])
                pair3 = (tri[0], tri[2])
                if pair1 not in got and pair2 not in got and pair3 not in got:
                    got.update((pair1, pair2, pair3))
                    cal.append(tri)
            else:
                cal = harpCal
        self.__level = N
        self.__dim = n
        self.__msgLen = len(cal) # (n,2) //3 = n!/(n-2)!/2 //3/def rot
        self.__cal = list(list(_) for _ in cal)
        self.__sgns = [1] * (self.__msgLen) # Key results

    def __str__(self):
        return str({"level": self.__level, "dimension": self.__dim,
                    "msg.len": self.__msgLen})

    def rotate(self, array):
        """Pass pairs of dimensions to rotate the code."""
        maxRot = self.__msgLen * 3 // 2 + 1
        self.__checkType(array, (list, tuple), "rotate", length=(1, maxRot))
        list(self.__checkType(_, (list, tuple), "rotate", length=2) for _ in array)
        list(self.__checkType(_[0], int, "rotate", (1, self.__dim)) for _ in array)
        list(self.__checkType(_[1], int, "rotate", (1, self.__dim)) for _ in array)
        self.__rotate(array)

    def __rotate(self, array):
        for pair in array:
            for tri in enumerate(self.__cal): # Rotate calibration structure
                sgn = self.__sgns[tri[0]]
                rot = tri[1][:]
                for val in enumerate(pair):
                    if val[1] in tri[1]:
                        idx = tri[1].index(val[1])
                        rot[idx] = pair[1] - val[0]
                    if val[0]:
                        sgn = -sgn # Change signs for 2nd rot. index
                sgn = self.__sort(rot, sgn)
                self.__cal[tri[0]] = rot
                self.__sgns[tri[0]] = sgn
        self.__sort(self.__cal, 1, self.__sgns)

    def reverse(self, array):
        """Return the array for rotate to decrypt instead of encrypt signs."""
        maxRot = self.__msgLen * 3 // 2 + 1
        self.__checkType(array, (list, tuple), "reverse", length=(1, maxRot))
        list(self.__checkType(_, (list, tuple), "reverse", length=2) for _ in array)
        return list(_[1], _[0] for _ in reversed(array))
```

```

def setCode(self, signs, cal=None):
    """Set signs and optionally the calibration structure."""
    self.__checkType(signs, (list, tuple), "setCode", length=self.__msgLen)
    list(self.__checkType(_, int, "setCode", (-1,1)) for _ in signs)
    if cal is not None:
        self.__checkType(cal, (list, tuple), "setCode", length=self.__msgLen)
        list(self.__checkType(_, (list,tuple), "setCode", length=3) for _ in cal)
        n = self.__dim
        for idx in (0, 1, 2):
            list(self.__checkType(_[idx], int, "setCode", (1, n)) for _ in cal)
        self.__setCode(signs, cal)
def __setCode(self, signs, cal):
    self.__sgns = list(signs)
    if cal is not None:
        self.__cal = list(cal)

def getCode(self):
    """Return signs and the calibration structure."""
    return (self.__sgns, self.__cal)

def __sort(self, array, sgn, signs=None):
    """Sort the list changing parity for pair swaps."""
    noMore = True
    while noMore:
        # Bubble sort - needed to not expose rotations used
        noMore = False
        for idx in range(1, len(array)):
            if array[idx -1] > array[idx]:
                tmp = array[idx -1]; array[idx -1] = array[idx]; array[idx] = tmp
                sgn = -sgn
                if signs:
                    tmp = signs[idx -1]; signs[idx -1] = signs[idx]; signs[idx] = tmp
                noMore = True
    return sgn

def comb(self, n, r, basis):
    """Yield the combinations of r in n elements for basis of length n."""
    self.__checkType(n, int, "comb", (1,0))
    self.__checkType(r, int, "comb", (0,n))
    self.__checkType(basis, (list, tuple), "comb", length=n)
    list(self.__checkType(_, int, "comb", (1, n)) for _ in basis)
    return self.__comb(n, r, basis)
def __comb(self, n, r, basis):
    if r > n //2:
        rng = range(1, len(basis) +1)
        for elem in reversed(list(self.__perm(n, [1] *(n -r), 1))):
            yield tuple(basis[idx -1] for idx in rng if idx not in elem)
    else:
        for elem in self.__perm(n, [1] *r, 1):
            yield tuple(basis[idx -1] for idx in elem)

def __perm(self, n, arr, offset):
    """Return permutation or combination arrays. The arr needs to be set to a
    list of one. Set offset to 0 to get permutations or 1 for combinations
    instead."""
    if offset < 0 or n < 0 or len(arr) > n:
        raise Exception("Invalid parameter for perm or comb: %s" %n)
    if len(arr) == 0:
        yield []
    else:
        for recuse in range(offset if offset else 1, n +1):
            if len(arr) > 1:
                for more in self.__perm(n, arr[1:], recuse if offset else 0):
                    arr = [recuse] +more
                    dup = False
                    for idx,elem in enumerate(arr):
                        if elem in arr[idx+1:]:
                            dup = True
                            break
                    if not dup:
                        yield arr
            else:
                yield [recuse]

```

```

def __checkType(self, arg, typ, method, size=[], length=None):
    """Raise exception if arg not the correct type in method for size."""
    if not isinstance(arg, typ):
        tmp = str(arg)
        if len(tmp) > 9:
            tmp = str(type(arg))
            raise Exception("Invalid parameter type (%s) for %s" %(tmp, method))
    if length:
        size = length
        arg = len(arg)
    if size:
        if isinstance(size, int):
            if size != arg:
                raise Exception("Invalid %s !=%d for %s" %(arg, size, method))
        elif isinstance(size, (list, tuple)):
            if len(size) == 2 and (arg < size[0] or \
                (size[1] > 0 and arg > size[1])):
                raise Exception("Invalid %s !in [%d,%s] for %s" %(arg, size[0],
                    size[1] if size[1] else "..", method))
        elif size:
            raise Exception("Invalid check %s parameters in %s" %(arg, method))

def seedRand(self):
    """The system does this by default."""
    random.seed(time.time())

def randRot(self, stage):
    """Generate random rotation pairs, stage=0: single for init, =1: odd
    dmensions only, =2: even dimensions, try for all, drop overlaps."""
    self.__checkType(stage, int, "randRot", (0,2))
    if stage == 0: # init case
        size = 2
        start = 1
        step = 1
    else:
        size = self.__msgLen *3 //2 +1
        start = 0 if stage == 1 else 2
        step = 2
    rots = []
    for pos in range(1, size):
        pair = [random.randrange(start, self.__dim, step),
            random.randrange(start, self.__dim, step)]
        if stage == 1: # Sender gets odd dimensions
            pair = [pair[0] +1, pair[1] +1]
        if pair[0] > pair[1]:
            tmp = pair[0]; pair[0] = pair[1]; pair[1] = tmp
        if pair not in rots and pair[0] != pair[1]:
            rots.append(pair)
    return rots

def pack(self, msg):
    """Change ASCII msg into 7 bits -> sign +1/-1 ints."""
    self.__checkType(msg, str, "pack", length=(1,0))
    if len(msg) *7 > self.__msgLen:
        raise Exception("Message it too long by %d character(s)" \
            %(len(msg) -self.__msgLen //7))
    signs = []
    for ch in msg:
        for bit in format(ord(ch), "07b"):
            signs.append(-1 if bit == "0" else 1)
    if len(signs) < self.__msgLen -7:
        for bit in format(0x25, "07b"): # Add control-d
            signs.append(-1 if bit == "0" else 1)
    for idx in range(len(signs), self.__msgLen): # pad with random bits
        signs.append(1 if random.randrange(2) else -1)
    return signs

def unpack(self, signs):
    """Change sign +1/-1 ints -> 7 bits for ASCII msg."""
    self.__checkType(signs, (list, tuple), "unpack", length=self.__msgLen)
    msg = ""
    for idx in range(0, len(signs) -6, 7):

```

```

        bits = ""
        for bit in signs[idx :idx +7]:
            bits += "1" if bit == 1 else "0"
        ch = chr(int(bits, 2))
        if ch == chr(0x25): # EOM control-d
            break
        msg += ch
    return msg

def save(self, struct, filename):
    """Save a python list with some type formatting of the structure."""
    nl = ""
    try:
        with open(filename, 'w') as fp:
            if isinstance(struct, (list, tuple, set)):
                fp.write("[\\n\\n")
                for val in struct:
                    if isinstance(val, (list, tuple, set)):
                        fp.write("%s %s,\\n" %(nl, val))
                        nl = ""
                    elif isinstance(val, str):
                        fp.write(" \\\"%s\\\",\\n" %val)
                        nl = ""
                    else:
                        fp.write(" %s,\\n" %val)
                        nl = "\\n"
                fp.write("]")
            elif isinstance(struct, str):
                fp.write("%s" %struct)
            else:
                fp.write("%s" %struct)
            fp.write("\\n")
    except BaseException as e:
        sys.stderr.write('%s: %s\\n' %(type(e).__name__, e))
def _generateSpace(self, store, parts, selfGetStrFn, selfSetStrFn,
                    status=None, pos=0, timeout=0):
    """Test code. Return new rotated 3- or 7-form calibrations as strings."""
    allRots = list((x,y) for x in range(1, self.__dim) \
                    for y in range(x +1, self.__dim +1))

    out = set()
    for offs,calStr in enumerate(store[parts[0] :parts[1]]):
        if status:
            status[pos] = offs
        cal = selfSetStrFn(calStr)
        for rot in allRots:
            self.__setCode(self.__sgns, cal)
            self.__rotate((rot,))
            newCal = selfGetStrFn()
            if newCal not in store:
                out.add(newCal)
        if timeout and time.time() >= time.time() + timeout:
            break
    return out

@staticmethod
def read(filename):
    """Read a Python structure (generally saved by save())."""
    try:
        with open(filename) as fp:
            return eval(fp.read())
    except Exception:
        typ,var,tb = sys.exc_info()
        raise Exception(typ(str(var).replace("\\\\\\\\\\\\\\\\", "\\\"")))

_CalCrypt = CalCrypt.__doc__ # Doco for calculators
#####
if __name__ == '__main__':
    import traceback
    class HelpException(Exception):
        """Don't report this exception."""
        pass
    try:

```

```

options = ("-v", "/v", "-vh", "-hv", "-h", "/?")
optIdx = (1,5) if sys.platform == "win32" else (0,4) # (v,h)
verbose = False
offset = stage = 0
while len(sys.argv) > 1 +offset and (sys.argv[1 +offset] in options):
    verbose = sys.argv[1 +offset] in options[:4]
    if sys.argv[1 +offset] in options[-4:]:
        help(CalCrypt)
        raise HelpException
    offset += 1
if len(sys.argv) > 2 +offset:
    stage = 1
    lev = int(sys.argv[1 +offset])
    base = sys.argv[2 +offset]
    msg = " ".join(sys.argv[3 +offset:])
    if not msg:
        raise Exception("Message is blank")
elif len(sys.argv) == 2 +offset:
    filename = sys.argv[1 +offset]
    stage, lev, base, signs, cal = CalCrypt.read(filename)
else:
    raise Exception("Invalid command line arguments")
crypt = CalCrypt(lev)
if stage == 1:
    crypt.seedRand()
    crypt.setCode(crypt.pack(msg))
    rot1 = crypt.randRot(2)
    crypt.rotate(rot1)
    crypt.save([rot1], base + "1.key")
    crypt.save([2, lev, base, *crypt.getCode()], base + "1.cry")
    crypt.save([rot1], base + "1.key")
    sys.stdout.write("Transmit file: %s1.cry (Key file: %s1.key)\n" %(base, base))
elif stage == 2:
    crypt.seedRand()
    crypt.setCode(signs, cal)
    rot2 = crypt.randRot(1)
    crypt.rotate(rot2)
    crypt.save([rot2], base + "2.key")
    crypt.save([3, lev, base, *crypt.getCode()], base + "2.cry")
    sys.stdout.write("Transmit file: %s2.cry (Key file: %s2.key)\n" %(base, base))
elif stage == 3:
    crypt.setCode(signs, cal)
    rot1 = CalCrypt.read(base + "1.key")[0]
    crypt.rotate(crypt.reverse(rot1))
    crypt.save([4, lev, base, *crypt.getCode()], base + "3.cry")
    sys.stdout.write("Transmit file: %s3.cry\n" %base)
elif stage == 4:
    crypt.setCode(signs, cal)
    rot2 = CalCrypt.read(base + "2.key")[0]
    crypt.rotate(crypt.reverse(rot2))
    msg = crypt.unpack(crypt.getCode()[0])
    crypt.save(msg, base + ".txt")
    sys.stdout.write("Message file: %s.txt\n" %base)
else:
    raise Exception("Corrupt transmitted file")
except Exception as e:
    cmd = ("py " if sys.platform == "win32" else "python[3] ") + "CalCrypt.py"
    print("Start usage: %s Level Basename \"Message\" (or unquoted)" %cmd)
    print("Other usage: %s \"Filename\"" %cmd)
    print("Help option: %s %s [%s]" %(cmd, options[optIdx[1]], options[optIdx[0]]))
    print("Desc: Start Level = 3..10 determines the maximum message length.")
    desc = ( \
        "Each usage creates a file to transmit to the other side and Other use",
        "processes the file with the last transmit being the decoded message.",
        "Message lengths are: 1, 5, 22, 93, 381, 1542, 6205, 24893 chars.")
    for txt in desc:
        print(" " +txt)
    if type(e) != HelpException:
        if verbose:
            traceback.print_exc()
        else:
            sys.stderr.write('%s: %s\n' %(type(e).__name__, e))

```

Appendix B - Test Code

```
#!/usr/bin/env python3
#####
## File: CalTest.py - test CalCrypt.py - G.P.Wilmot (c) Apr 2026
#####
import sys, os, datetime, time, argparse
from math import factorial as fact
import CalCrypt
from multiprocessing import Process
try:
    from multiprocessing import shared_memory
except:
    shared_memory = None

class CalTest():
    """Class to test quasi-calibration at level N=3..5 (realistically only 3/4)
    by generating all possible calibrations and finding unique quasi-algebras
    from 3-form calibration. Can also generate 7-form calcs for N=4."""
    memLimitMB = 500 # Abort if less mem than this

    def __init__(self, N, data="", hours=0, harpCal=None):
        """Create the initial calibration structure for this level."""
        self.dim = int(pow(2, N)) - 1
        self.subCal = 3 if harpCal is None else 7
        self.crypt = CalCrypt.CalCrypt(N, None if harpCal is None \
                                         else self.convertCalStr(harpCal))
        self.msgLen = len(self.crypt.getCode()[0])
        self.dataPath = data
        self.timeout = int(time.time() + hours * 3600) if hours else 0

        # PCode calc. from Structure of C-D: Table 5
        # 28, 252, 2156, 18732, 159852, 1331820, 10902892, 88310124
        self.__pc = ((1,0,0,0), (8,7,0,0), (50,63,42,0), (310,413,504,168),
                     (2046,2583,4158,3024), (14478,16905,30996,34776),
                     (107950,118251,229194,332640), (831470,873901,1729728,2912616))
        tmp = self.__pc[N-3]
        self.__uCode = tmp[0] * 28 + tmp[1] * 4 + tmp[2] * 12 + tmp[3] * 14

    def getUcode(self):
        """Return the P code for the n-D calibration, (eg 252 for sedenions)."""
        return self.__uCode

    def freeMemMB(self):
        """Return the size of free RAM in MB."""
        if sys.platform == "win32":
            process = os.popen('systeminfo 2>nul |find "Available Phys"')
            result = process.read()
            process.close()
            if len(result.split()) < 3:
                sys.stderr.write("Never trust Windows: %s\n", result)
                return self.freeMemMB
            return int(result.split()[3].replace(",",""))
        return os.sysconf('SC_AVPHYS_PAGES')//256

    def allSigns(self):
        """Yield half of all sign variations of the current calibration size."""
        stopCnt = -1
        halfStop = (self.msgLen % 2 == 0)
        halfDim = self.msgLen // 2
        halfComb = fact(self.msgLen)//fact(self.msgLen - halfDim) // fact(halfDim)
        buf = list(x for x in range(1, self.msgLen + 1))
        for n in range(halfDim + 1):
            for cnt, signs in enumerate(self.crypt.comb(self.msgLen, n, buf)):
                if n == halfDim and halfStop and cnt == halfComb:
                    break
                stopCnt -= 1
                if stopCnt == -1:
                    break
                yield signs
            if stopCnt == -1:
                break
```

```

def getPcode(self):
    """Run from Python to return the current Pcode. See getPcodes()."""
    signs,cal = self.crypt.getCode()
    signsGen = (list(idx +1 for idx,val in enumerate(signs) if val < 0),)
    return self.__getPcodes(cal, signsGen)[0]

def getPcodes(self, cal, signsGen, loopSize, firstCal=False):
    """Return the P codes for cal with all minus signs in signsGen up to
    loopSize of firstCal (if set). Turns the cal 3-forms into a
    multiplication table and counts all ordered non-associative triads
    that are unique of Type B & X, [a,b,c]!=0, a<b<c."""
    return self.__getPcodes(cal, signsGen, loopSize, firstCal)
def __getPcodes(self, cal, signsGen, loopSize=0, firstCal=False):
    pairs = []
    for tri in cal:
        pairs.extend([((tri[0], tri[1]), tri[2]),
            ((tri[1], tri[2]), tri[0]), ((tri[2], tri[0]), tri[1]))])
    table = dict(pairs) # Mult. table as dimension pairs = positive of triad
    codes = []
    for idx,negSigns in enumerate(signsGen):
        for sign in negSigns:
            if sign:
                neg = cal[sign -1]
                table[(neg[0], neg[1])] *= -1
                table[(neg[1], neg[2])] *= -1
                table[(neg[2], neg[0])] *= -1
        cnt = 0
        for x in range(1, self.dim -1):
            for y in range(x +1, self.dim):
                for z in range(y +1, self.dim +1):
                    if [x, y, z] not in cal:
                        m1 = table[(x, y)] if (x, y) in table else -table[(y, x)]
                        sm1 = 1 if m1 > 0 else -1
                        m1 = abs(m1)
                        m2 = (table[(m1, z)] if (m1, z) in table else -table[(z, m1)])
                        m3 = table[(y, z)] if (y, z) in table else -table[(z, y)]
                        sm3 = 1 if m3 > 0 else -1
                        m3 = abs(m3)
                        m4 = (table[(x, m3)] if (x, m3) in table else -table[(m3, x)])
                        if m2 *sm1 != m4 *sm3:
                            cnt += 1
        codes.append(cnt)
        if idx == loopSize:
            break
        if firstCal and codes[-1] == self.getUcode():
            break
    for sign in negSigns:
        neg = cal[sign -1]
        table[(neg[0], neg[1])] *= -1
        table[(neg[1], neg[2])] *= -1
        table[(neg[2], neg[0])] *= -1
    return codes

def generateSpaceTasking(self, selfGetStrFn, selfSetStrFn, basename,
    store=None, cpus=1, lastCal=0, dump=False):
    """Return a list of all 3- or 7-form calibrations as strings.
    Packing into hex strings can only be done with N=3/4."""
    lastSize = lastCal
    status = oldStatus = None
    if shared_memory:
        status = shared_memory.ShareableList([0] *cpus)
    else:
        sys.stderr.write("Shared memory is not available for status logging\n")
    if store is None:
        store = [selfGetStrFn()]
    running = True
    while lastSize < len(store) and running:
        out = set()
        if cpus == 1 or len(store) -lastSize < cpus:
            out = self._generateSpace(selfGetStrFn, selfSetStrFn, store,
                (lastSize,len(store)))

```

```

else:
    size = (len(store) -lastSize) //cpus +1
    taskParts= []
    accum = lastSize
    for _ in range(cpus -(size *(cpus +1) -len(store) +lastSize) //size +1):
        taskParts.append((accum, accum +size))
        accum += size
    if dump:
        sys.stdout.write("%s: Free %dMB Task sizes %d-%d <= [%d] *%d \n" \
            %(datetime.datetime.today(), self.freeMemMB(), len(store),
              lastSize, taskParts[0][1] -taskParts[0][0], len(taskParts)))
    process = []
    for pos,parts in enumerate(taskParts):
        process.append(Process(target=self._generateSpace, args=(selfGetStrFn,
            selfSetStrFn, store, parts, basename +"%d.tmp" %pos,
            status, pos)))
    for proc in process:
        proc.start()
    time.sleep(10)
    for pos,task in enumerate(process):
        while running and task.is_alive():
            try:
                if dump:
                    tmp = list(status[idx] for idx in range(cpus)) if status else None
                    if tmp != oldStatus or pos == 0:
                        sys.stdout.write("%s: Free %dMB Status: %s\n" \
                            %(datetime.datetime.today(), self.freeMemMB(), tmp))
                        sys.stdout.flush()
                        oldStatus = tmp
                    if self.freeMemMB() < self.memLimitMB:
                        sys.stderr.write("Aborting due to memory limit\n")
                        store.insert(0, "Abort: Due to memory limit")
                        running = False
                else:
                    task.join(1800)
            except KeyboardInterrupt:
                sys.stderr.write("Aborting due to keyboard interrupt\n")
                store.insert(0, "Abort: Due to keyboard interrupt")
                sys.stderr.flush()
                running = False
        else:
            if task.is_alive():
                task.terminate()
            else:
                task.join()
    time.sleep(10)
    if dump:
        sys.stdout.write("Read files %s[0..%d].tmp\n" %(basename,
            len(process) -1))
        sys.stdout.flush()
    tmpName = (os.path.join(self.dataPath, basename) \
        if self.dataPath else basename) +"%d.tmp"
    for pos in range(len(process)):
        out.update(self.crypt.read(tmpName %pos))
    for pos in range(len(process)):
        os.remove(tmpName %pos)
    sys.stderr.write("Processed Size %d + adding %d = %d\n" %(len(store) \
        -lastSize, len(out), len(store) +len(out)))
    lastSize = len(store)
    store.extend(out)
    if self.timeout and self.timeout < time.time():
        sys.stderr.write("Processing stopped due to timeout\n")
        break
    if dump:
        sys.stdout.write("%s: Free %dMB\n" %(datetime.datetime.today(),
            self.freeMemMB()))
return store

```

```

def _generateSpace(self, selfGetStrFn, selfSetStrFn, store, parts,
                    filename=None, status=None, pos=0):
    """Internal method to return a list of all calibrations as strings."""
    try:
        out = self.crypt._generateSpace(store, parts, selfGetStrFn, selfSetStrFn,
                                         status, pos, self.timeout)
    except KeyboardInterrupt:
        out = set()
    except Exception as e:
        out = "%s: %s\n" % (type(e).__name__, e)
        sys.stderr.write(out)
    if filename:
        self.crypt.save(out, os.path.join(self.dataPath, filename) \
                        if self.dataPath else filename)
    if status:
        status[pos] = -1
    return out

def generateQuasiCalsTasking(self, allCals, basename, store=[], cpus=1,
                             dump=False):
    """Return a list of P code list for first quasi-cal found, its sign
    index list, first sedenion index within this calibration & calibration
    string then the total count of quasi-calibrations. The sign index is zero
    for all positive (-1 if not found) else the index of a single minus sign.
    The P code is the count of all Type B/X Non Associative triads."""
    quasiCodes = []
    struct = []
    if cpus == 1 or len(allCals) < cpus:
        store.extend(self._generateQuasiCals(allCals, (0, len(allCals))))
    else:
        size = len(allCals) // cpus + 1
        taskParts = []
        accum = 0
        for _ in range(cpus - (size * (cpus + 1) - len(allCals)) // size + 1):
            taskParts.append((accum, accum + size))
            accum += size
        sys.stdout.write("%s: Free %dMB Task sizes %d <= [%d] *%d \n" \
                        % (datetime.datetime.today(), self.freeMemMB(), len(allCals),
                           taskParts[0][1] - taskParts[0][0], len(taskParts)))
        status = oldStatus = None
        if shared_memory:
            status = shared_memory.ShareableList([0] * cpus)
        else:
            sys.stderr.write("Shared memory is not available for status logging\n")
        process = []
        for pos, parts in enumerate(taskParts):
            process.append(Process(target=self._generateQuasiCals, args=(allCals,
                               parts, basename + "%d.tmp" % pos, status, pos)))
        for proc in process:
            proc.start()
            time.sleep(10)
            running = True
        for pos, task in enumerate(process):
            while running and task.is_alive():
                try:
                    tmp = list(status[idx] for idx in range(cpus)) if status else None
                    if tmp != oldStatus or pos == 0:
                        if dump:
                            sys.stdout.write("%s: Free %dMB Status: %s\n" \
                                              % (datetime.datetime.today(), self.freeMemMB(), tmp))
                            sys.stdout.flush()
                            oldStatus = tmp
                    if self.freeMemMB() < self.memLimitMB:
                        sys.stderr.write("Aborting due to memory limit\n")
                        store.insert(0, "Abort: Due to memory limit")
                        running = False
                    else:
                        task.join(1800)
                except KeyboardInterrupt:
                    sys.stderr.write("Aborting due to keyboard interrupt\n")
                    store.insert(0, "Abort: Due to keyboard interrupt")
                    running = False

```

```

        else:
            if task.is_alive():
                task.terminate()
            else:
                task.join()
    time.sleep(10)
    if dump:
        sys.stdout.write("Read files %s[0..%d].tmp\n" %(basename,
                                                                len(process) -1))

        sys.stdout.flush()
        tmpName = (os.path.join(self.dataPath, basename) \
                    if self.dataPath else basename) + "%d.tmp"
        for pos in range(len(process)): # Read task file results
            store.extend(self.crypt.read(tmpName %pos))
        for pos in range(len(process)):
            os.remove(tmpName %pos)
    for st in store: # Merge results with other tasks/runs
        tmp = {}
        for idx,pCode in enumerate(st[0]):
            if pCode not in quasiCodes:
                quasiCodes.append(pCode)
                tmp[pCode] = st[1][idx]
        if tmp:
            struct.append((list(tmp.keys()), list(tmp.values()), st[2], st[3]))
    if dump:
        sys.stdout.write("%s: Free %dMB\n" %(datetime.datetime.today(),
                                            self.freeMemMB()))
    if self.timeout and self.timeout < time.time():
        sys.stderr.write("Processing stopped due to timeout\n")
    return struct, len(quasiCodes)

def _generateQuasiCals(self, allCals, parts, filename=None, status=None,
                        pos=0):
    """Internal method to return unique P codes, structure and their count."""
    struct = []
    quasiCodes = []
    ultCode = self.crypt.getUcode()
    try:
        for offs,calStr in enumerate(allCals[parts[0] :parts[1]]):
            if status:
                status[pos] = offs
            cal = self.convertCalStr(calStr)
            codes = self.crypt.getPcodes(cal, self.allSigns(),
                                         len(cal) *(len(cal) +3)/2, self.dim > 7) # Up to 2 minus signs
            quiIdx = []
            quiCode = []
            ultIdx = -2
            for idx,cod in enumerate(codes):
                if cod != ultCode and cod not in quasiCodes:
                    quiIdx.append(idx)
                    quiCode.append(cod)
                    quasiCodes.append(cod)
                    if ultIdx < 0:
                        ultIdx = codes.index(ultCode) if ultCode in codes else -1
            if ultIdx >= -1:
                struct.append([quiCode, quiIdx, ultIdx, calStr])
            if self.timeout and self.timeout < time.time():
                break
    except KeyboardInterrupt:
        pass
    if filename:
        self.crypt.save(struct, os.path.join(self.dataPath, filename) \
                        if self.dataPath else filename)

    if status:
        status[pos] = -1
    return struct

def getCal3Str(self):
    """Return the N=3 calibration) packed as 3 hex chars."""
    sgns,cal = self.crypt.getCode()
    strs = list("%X%X%X" %tuple(x) for x in cal)
    return "".join(strs)

```

```

def getCal7Str(self):
    """Test code. Return the N=4 calibration packed as 7 hex chars."""
    sgns,cal = self.crypt.getCode()
    strs = list("%X%X%X%X%X%X%X" %tuple(x) for x in cal)
    return "".join(strs)

def convertCalStr(self, str):
    """Test code. Return a string cal. as a list of dimensions. Only for N=3/4."""
    cal = []
    for x in range(0, len(str), self.subCal):
        cal.append(list(int(x, 16) for x in str[x :x +self.subCal]))
    return cal

def saveAll(self, arr, fileBase):
    """Save calibrations in chunks of 2M lines in each file."""
    sizeFile = 2000000
    sys.stdout.write("Output files %s[1..%d].txt for %d calibrations\n" \
        %(fileBase, len(arr) //sizeFile +1, len(arr)))
    offset = 0
    for idx in range(0, len(arr), sizeFile):
        offset += 1
        self.crypt.save(arr[idx: idx +sizeFile], (os.path.join(self.dataPath,
            fileBase) if self.dataPath else fileBase) +str(offset) +".txt")

def readLastCals(self, baseName, dump):
    """Read last cal. output files."""
    out = []
    fileBase = os.path.join(self.dataPath, baseName) \
        if self.dataPath else baseName
    cnt = 1
    if not os.path.isfile(fileBase +"%d.txt" %cnt):
        raise Exception("File does not exist: %s" %(fileBase +"%d.txt" %cnt))
    if dump:
        sys.stdout.write("Read %s: " %(fileBase +"?.txt files"))
    while os.path.isfile(fileBase +"%d.txt" %cnt):
        if dump:
            sys.stdout.write("%d " %cnt)
            sys.stdout.flush()
        try:
            with open(fileBase +"%d.txt" %cnt) as fp:
                out.extend(eval(fp.read()))
        except Exception:
            typ,var,tb = sys.exc_info()
            raise Exception(typ(str(var).replace("\\\\\\\\\\\\", "\\")))
        cnt += 1
    if dump:
        sys.stdout.write("\n")
    return out

def readLastQuasis(self, baseName, dump):
    """Read last quasi. output file."""
    out = []
    fileBase = os.path.join(self.dataPath, baseName) \
        if self.dataPath else baseName
    if not os.path.isfile(fileBase +"0.txt"):
        raise Exception("File does not exist: %s0.txt" %fileBase)
    if dump:
        sys.stdout.write("Read %s0.txt\n" %fileBase)
    try:
        with open(fileBase +"0.txt") as fp:
            out.extend(eval(fp.read()))
    except Exception:
        typ,var,tb = sys.exc_info()
        raise Exception(typ(str(var).replace("\\\\\\\\\\\\", "\\")))
    return out

_CalTest = CalTest.__doc__ # Doco for calculators

```

```
#####
if __name__ == '__main__':
    import traceback
    class HelpException(Exception):
        """Don't report this exception."""
        pass

    # Get command line parameter and options
    desc = """Generate all primary calibrations and find first of each quasi-cals
    as 3-form calibrations for Level=3/4 with dimensions 7/15, resp.
    Level=7 is code for 15-D 7-form calibration, generate only.
    Levels 4/7 take 3 days so needs 100 cpus on the Phoenix HPC.
    Output is <Filebase>3/7cals?.txt are the primary quasi-calibrations
    in 20 million line chunks (?>0) and Level 3/4 ?=0 is quasi-cal P codes,
    sign index for first quasi-cals, first cal index & hex string cal."""
    parser = argparse.ArgumentParser("CatTest.py", description=desc)
    parser.add_argument("level", type=int, default = None,
        help="Level = 3/4/7 determines the calibration size and type")
    parser.add_argument("-f", "--fullHelp", action='store_true',
        help="Print full documentation")
    parser.add_argument("-v", "--verbose", action='store_true',
        help="Enable traceback and logging")
    parser.add_argument("-p", "--phase", choices=(1,2,3,4), type=int, default=3,
        help="1: generate cals, 2: find quasi-cals, 3: both, 4: redo 3")
    parser.add_argument("-c", "--cpus", type=int, default=1,
        help="Multiprocess tasks (default 1)")
    parser.add_argument("-d", "--data", type=str, default="",
        help="Output file path (default none)")
    parser.add_argument("-b", "--baseName", type=str, default = "cal",
        help="Output file prefix name (default cal)")
    parser.add_argument("-t", "--timeout", type=int, default=0,
        help="Stop after these hours (default no stop)")
    parser.add_argument("-l", "--lastCal", type=int, default=0,
        help="Restart from this index in cals file. Use p4")
    args = parser.parse_args()
    try:
        if args.fullHelp:
            help(CalTest)
            raise HelpException

        if args.data and not os.path.isdir(args.data):
            raise Exception("Data path %s does not exist" %args.data)
        if args.cpus < 1:
            raise Exception("Invalid number of cpus")

        if args.level in (3, 4, 5):
            test = CalTest(args.level, args.data,
                args.timeout *(2/3 if args.phase == 4 else 1))
            if args.level == 5: # Need get/setStr for 31-D ie chars G..U
                raise Exception("Level 5 not implemented yet")
            if args.phase in (1, 3, 4):
                if args.phase == 4:
                    allCals = test.readLastCals(args.baseName + "%dcals" %args.level,
                        args.verbose)
                else:
                    allCals = None
                    allCals = test.generateSpaceTasking(test.getCal3Str,
                        test.convertCalStr, args.baseName + "%dcals" %args.level,
                        allCals, args.cpus, args.lastCal, args.verbose)
                    test.saveAll(allCals, args.baseName + "%dcals" %args.level)
            else:
                allCals = test.readLastCals(args.baseName + "%dcals" %args.level,
                    args.verbose)
            if allCals[0][:6] == "Abort:":
                print("Not all quasi-sedenions will be extracted")
                allCals = allCals[1:]
            if args.phase in (2, 3, 4):
                if args.phase == 4:
                    if args.timeout: # Redo - Phase 1 takes twice as long
                        test.timeout = int(time.time() +args.timeout *3600 /3)
                    store = test.readLastQuasis(args.baseName + "%dcals" %args.level,
                        args.verbose)
```

```

else:
    store = []
    print("Now extract unique quasi-cals and first cal. (P%d) %s" \
          %(test.crypt.getUcode(), "indices"))
    struct, quiCnt = test.generateQuasiCalsTasking(allCals[args.lastCal:],
          args.baseName + "%dcals" %args.level,
          store, args.cpus, args.verbose)
    print("Output file %s for cals reps and %d quasi-cals" \
          %(args.baseName + "%dcals0.txt" %args.level, quiCnt))
    test.crypt.save(struct, (os.path.join(args.data, args.baseName) \
          if args.data else args.baseName) + "%dcals0.txt" %args.level)

elif args.level == 7:
    cal15 = "123456712389AB123CDEF14589CD145ABEF16789EF167ABCD246" \
            + "8ACE2469BDF2578ADF2579BCE3478BCF3479ADE3568BDE3569ACF"
    test = CalTest(4, args.data, args.timeout, cal15)
    dim = int(2 ** 4) - 1
    if args.phase == 4:
        allCals = test.readAllSpin(args.baseName + "7cals", args.verbose)
    else:
        allCals = None
    allCals = test.generateSpaceTasking(test.getCal7Str, test.convertCalStr,
          args.baseName + "7cals", allCals, args.cpus, args.lastCal,
          args.verbose)
    test.saveAll(allCals, args.baseName + "7cals")
else:
    raise Exception("Invalid level not in (3..5,7)")
except Exception as e:
    if args.verbose:
        traceback.print_exc()
    else:
        sys.stderr.write('%s: %s\n' %(type(e).__name__, e))

```

Appendix C - 100 Sedenion Quasi-Calibrations

Table 4: Sedenion Quasi-Calibrations

$\mathbb{P}_{122}$	$\{e_{123} + e_{149} + e_{15E} + e_{16D} - e_{17B} + e_{18A} + e_{1CF} + e_{24B} + e_{25C} + e_{268} + e_{279} + e_{2AD} + e_{2EF} + e_{347} + e_{35F} + e_{36A} - e_{38D} + e_{39B} + e_{3CE} + e_{456} + e_{48C} + e_{4AF} + e_{4DE} + e_{57A} + e_{58B} + e_{59D} + e_{67F} + e_{69E} + e_{6BC} + e_{78E} + e_{7CD} + e_{89F} + e_{9AC} + e_{ABE} + e_{BDF}$
$\mathbb{P}_{124}$	$e_{e12C} + e_{138} - e_{14F} + e_{15E} + e_{16D} + e_{17A} + e_{19B} + e_{23B} + e_{24E} + e_{25F} + e_{267} + e_{289} + e_{2AD} + e_{34A} + e_{356} + e_{37F} + e_{39C} + e_{3DE} + e_{45C} + e_{469} + e_{478} + e_{4BD} + e_{57B} + e_{58D} + e_{59A} + e_{68E} - e_{6AC} + e_{6BF} + e_{79E} + e_{7CD} + e_{8AF} + e_{8BC} + e_{9DF} + e_{ABE} + e_{CEF}$
$\mathbb{P}_{126}$	$e_{12B} + e_{135} + e_{14D} + e_{167} + e_{189} + e_{1AC} + e_{1EF} + e_{239} + e_{246} + e_{258} + e_{27D} + e_{2AF} + e_{2CE} + e_{34F} + e_{36A} - e_{37C} + e_{38B} + e_{3DE} + e_{45E} + e_{47B} + e_{48C} + e_{49A} + e_{56C} + e_{57A} + e_{59B} + e_{5DF} + e_{68E} - e_{69F} + e_{6BD} + e_{78F} + e_{79E} + e_{8AD} + e_{9CD} + e_{ABE} + e_{BCF}$
$\mathbb{P}_{128}$	$e_{129} + e_{13B} + e_{14A} - e_{15F} + e_{16E} + e_{17D} + e_{18C} + e_{234} + e_{25C} + e_{267} + e_{28F} + e_{2AB} + e_{2DE} + e_{35E} + e_{36F} + e_{378} + e_{39A} + e_{3CD} + e_{45D} + e_{468} + e_{47F} + e_{49B} + e_{4CE} + e_{56B} + e_{57A} + e_{589} + e_{69D} + e_{6AC} + e_{79E} + e_{7BC} + e_{8AE} + e_{8BD} + e_{9CF} + e_{ADF} + e_{BEF}$
$\mathbb{P}_{130}$	$e_{12A} + e_{13B} + e_{149} - e_{15F} + e_{16E} + e_{17D} + e_{18C} + e_{234} + e_{25D} + e_{268} + e_{27F} + e_{29B} + e_{2CE} + e_{35E} + e_{36F} + e_{378} + e_{39A} + e_{3CD} + e_{45C} + e_{467} + e_{48F} + e_{4AB} + e_{4DE} + e_{56B} + e_{57A} + e_{589} + e_{69D} + e_{6AC} + e_{79E} + e_{7BC} + e_{8AE} + e_{8BD} + e_{9CF} + e_{ADF} + e_{BEF}$
$\mathbb{P}_{132}$	$e_{12A} + e_{139} + e_{14B} - e_{15F} + e_{16E} + e_{17D} + e_{18C} + e_{234} + e_{25D} + e_{268} + e_{27F} + e_{29B} + e_{2CE} + e_{35C} + e_{367} + e_{38F} + e_{3AB} + e_{3DE} + e_{45E} + e_{46F} + e_{478} + e_{49A} + e_{4CD} + e_{56B} + e_{57A} + e_{589} + e_{69D} + e_{6AC} + e_{79E} + e_{7BC} + e_{8AE} + e_{8BD} + e_{9CF} + e_{ADF} + e_{BEF}$
$\mathbb{P}_{134}$	$e_{129} + e_{13D} + e_{14C} + e_{15F} + e_{167} + e_{18A} + e_{1BE} + e_{234} + e_{256} + e_{27F} + e_{28E} + e_{2AB} + e_{2CD} + e_{35B} + e_{36A} + e_{378} + e_{39C} + e_{3EF} + e_{45A} + e_{46B} + e_{47E} - e_{48F} + e_{49D} + e_{579} + e_{58C} + e_{5DE} + e_{68D} + e_{69F} + e_{6CE} + e_{7AD} + e_{7BC} + e_{89B} + e_{9AE} + e_{ACF} + e_{BDF}$
$\mathbb{P}_{136}$	$e_{125} + e_{138} + e_{14B} + e_{169} + e_{17C} + e_{1AD} + e_{1EF} + e_{23F} + e_{249} + e_{26B} + e_{27D} + e_{28E} + e_{2AC} + e_{34C} + e_{35E} + e_{36D} - e_{37B} + e_{39A} + e_{456} + e_{478} + e_{4AF} + e_{4DE} + e_{57A} + e_{58F} + e_{59B} + e_{5CD} + e_{67F} + e_{68A} + e_{6CE} + e_{79E} + e_{89D} + e_{8BC} + e_{9CF} + e_{ABE} + e_{BDF}$
$\mathbb{P}_{138}$	$e_{129} + e_{138} - e_{14F} + e_{15E} + e_{16C} + e_{17B} + e_{1AD} + e_{23D} + e_{24B} + e_{256} + e_{27F} + e_{28A} + e_{2CE} + e_{34C} + e_{357} + e_{36F} + e_{39A} + e_{3BE} + e_{45A} + e_{468} + e_{479} + e_{4DE} + e_{58B} + e_{59C} + e_{5DF} + e_{67D} + e_{69E} + e_{6AB} + e_{78E} + e_{7AC} + e_{89D} + e_{8CF} + e_{9BF} + e_{AEF} + e_{BCD}$
$\mathbb{P}_{140}$	$e_{12B} + e_{135} + e_{14D} + e_{167} + e_{189} + e_{1AC} + e_{1EF} + e_{239} + e_{246} + e_{258} + e_{27D} + e_{2AF} + e_{2CE} + e_{34F} + e_{36A} - e_{37C} + e_{38B} + e_{3DE} + e_{45E} + e_{47B} + e_{48C} + e_{49A} + e_{56C} + e_{57A} + e_{59B} + e_{5DF} + e_{68E} + e_{69F} + e_{6BD} + e_{78F} + e_{79E} + e_{8AD} + e_{9CD} + e_{ABE} + e_{BCF}$
$\mathbb{P}_{142}$	$e_{128} + e_{139} - e_{14F} + e_{15E} + e_{16D} + e_{17C} + e_{1AB} + e_{23B} + e_{24C} + e_{256} + e_{27F} + e_{29A} + e_{2DE} + e_{34D} + e_{357} + e_{36F} + e_{38A} + e_{3CE} + e_{45A} + e_{469} + e_{478} + e_{4BE} + e_{58D} + e_{59C} + e_{5BF} + e_{67B} + e_{68E} + e_{6AC} + e_{79E} + e_{7AD} + e_{89B} + e_{8CF} + e_{9DF} + e_{AEF} + e_{BCD}$
$\mathbb{P}_{144}$	$e_{123} + e_{14E} + e_{15F} + e_{167} + e_{189} + e_{1AB} + e_{1CD} + e_{24D} + e_{257} + e_{26F} + e_{28A} + e_{29B} + e_{2CE} + e_{34C} + e_{356} + e_{37F} + e_{38B} + e_{39A} + e_{3DE} + e_{45A} + e_{469} + e_{478} + e_{4BF} + e_{58D} + e_{59C} - e_{5BE} + e_{68E} + e_{6AC} + e_{6BD} + e_{79E} + e_{7AD} + e_{7BC} + e_{8CF} + e_{9DF} + e_{AEF}$
$\mathbb{P}_{146}$	$e_{123} + e_{14A} + e_{15B} + e_{167} + e_{189} + e_{1CF} + e_{1DE} + e_{246} + e_{258} + e_{27A} + e_{29B} + e_{2CD} + e_{2EF} + e_{347} + e_{359} + e_{36A} + e_{38B} + e_{3CE} + e_{3DF} + e_{45F} + e_{48E} + e_{49D} + e_{4BC} + e_{56E} + e_{57D} + e_{5AC} + e_{68F} + e_{69C} + e_{6BD} - e_{78C} + e_{79F} + e_{7BE} + e_{8AD} + e_{9AE} + e_{ABF}$
$\mathbb{P}_{148}$	$e_{123} + e_{145} + e_{16F} + e_{17C} + e_{189} + e_{1AB} + e_{1DE} + e_{24E} + e_{25D} - e_{26C} + e_{27F} + e_{28B} + e_{29A} + e_{34D} + e_{35E} + e_{367} + e_{38A} + e_{39B} + e_{3CF} + e_{46B} + e_{479} + e_{48C} + e_{4AF} + e_{56A} + e_{578} + e_{59C} + e_{5BF} + e_{68E} + e_{69D} + e_{7AE} + e_{7BD} + e_{8DF} + e_{9EF} + e_{ACD} + e_{BCE}$
$\mathbb{P}_{150}$	$e_{123} + e_{145} + e_{168} + e_{179} + e_{1AB} + e_{1CD} + e_{1EF} + e_{249} + e_{257} + e_{26B} + e_{28A} + e_{2CE} + e_{2DF} + e_{347} + e_{359} + e_{36A} + e_{38B} + e_{3CF} + e_{3DE} + e_{46E} + e_{48F} + e_{4AD} + e_{4BC} + e_{56F} + e_{58E} + e_{5AC} + e_{5BD} + e_{67D} + e_{69C} - e_{78C} + e_{7AE} + e_{7BF} + e_{89D} + e_{9AF} + e_{9BE}$
$\mathbb{P}_{152}$	$e_{123} + e_{14E} + e_{15F} + e_{167} + e_{189} + e_{1AB} + e_{1CD} + e_{24D} + e_{257} + e_{26F} + e_{28A} + e_{29B} + e_{2CE} + e_{34C} + e_{356} + e_{37F} + e_{38B} + e_{39A} + e_{3DE} + e_{45A} + e_{469} + e_{478} + e_{4BF} + e_{58D} + e_{59C} + e_{5BE} + e_{68E} + e_{6AC} - e_{6BD} + e_{79E} + e_{7AD} + e_{7BC} + e_{8CF} + e_{9DF} + e_{AEF}$
$\mathbb{P}_{154}$	$e_{123} + e_{145} + e_{168} + e_{179} + e_{1AB} + e_{1CD} + e_{1EF} + e_{249} + e_{257} + e_{26B} + e_{28A} + e_{2CE} + e_{2DF} + e_{347} + e_{359} + e_{36A} + e_{38B} + e_{3CF} + e_{3DE} + e_{46F} + e_{48E} + e_{4AC} + e_{4BD} + e_{56E} + e_{58F} + e_{5AD} + e_{5BC} + e_{67C} + e_{69D} - e_{78D} + e_{7AF} + e_{7BE} + e_{89C} + e_{9AE} + e_{9BF}$
$\mathbb{P}_{156}$	$e_{123} + e_{145} + e_{168} + e_{17A} + e_{19B} + e_{1CD} + e_{1EF} + e_{247} + e_{25A} + e_{269} + e_{28B} + e_{2CF} + e_{2DE} + e_{34A} + e_{357} + e_{36B} + e_{389} + e_{3CE} + e_{3DF} + e_{46E} + e_{48F} + e_{49D} + e_{4BC} + e_{56F} + e_{58E} + e_{59C} + e_{5BD} + e_{67D} + e_{6AC} - e_{78C} + e_{79E} + e_{7BF} + e_{8AD} + e_{9AF} + e_{ABE}$
$\mathbb{P}_{158}$	$e_{123} + e_{145} + e_{167} + e_{18A} + e_{19B} + e_{1CD} + e_{1EF} + e_{248} + e_{25A} + e_{269} + e_{27B} + e_{2CE} + e_{2DF} + e_{34A} + e_{358} + e_{36B} + e_{379} + e_{3CF} + e_{3DE} + e_{46F} + e_{47E} + e_{49D} + e_{4BC} - e_{56E} + e_{57F} + e_{59C} + e_{5BD} + e_{68D} + e_{6AC} + e_{78C} + e_{7AD} + e_{89F} + e_{8BE} + e_{9AE} + e_{ABF}$
$\mathbb{P}_{160}$	$e_{123} + e_{14E} + e_{15F} + e_{167} + e_{189} + e_{1AB} + e_{1CD} + e_{24D} + e_{257} + e_{26F} + e_{28A} - e_{29B} + e_{2CE} + e_{34C} + e_{356} + e_{37F} + e_{38B} + e_{39A} + e_{3DE} + e_{45A} + e_{469} + e_{478} + e_{4BF} + e_{58D} + e_{59C} + e_{5BE} + e_{68E} + e_{6AC} + e_{6BD} + e_{79E} + e_{7AD} + e_{7BC} + e_{8CF} + e_{9DF} + e_{AEF}$
$\mathbb{P}_{162}$	$e_{123} + e_{14A} + e_{15B} + e_{167} + e_{189} + e_{1CD} + e_{1EF} + e_{246} + e_{258} + e_{27A} + e_{29B} + e_{2CE} + e_{2DF} + e_{347} + e_{359} + e_{36A} + e_{38B} + e_{3CF} + e_{3DE} + e_{45E} + e_{48C} + e_{49D} - e_{4BF} + e_{56C} + e_{57D} + e_{5AF} + e_{68E} + e_{69F} + e_{6BD} + e_{78F} + e_{79E} + e_{7BC} + e_{8AD} + e_{9AC} + e_{ABE}$
$\mathbb{P}_{164}$	$e_{123} + e_{145} + e_{16B} + e_{17A} + e_{189} + e_{1CD} + e_{1EF} + e_{24A} + e_{257} + e_{268} + e_{29B} + e_{2CE} + e_{2DF} + e_{347} + e_{35A} + e_{369} + e_{38B} + e_{3CF} + e_{3DE} + e_{46E} + e_{48C} + e_{49D} - e_{4BF} + e_{56F} + e_{58D} + e_{59C} + e_{5BE} + e_{67D} + e_{6AC} + e_{78F} + e_{79E} + e_{7BC} + e_{8AE} + e_{9AF} + e_{ABD}$
$\mathbb{P}_{166}$	$e_{128} + e_{139} + e_{14E} + e_{15F} + e_{16C} + e_{17D} + e_{1AB} + e_{23A} + e_{246} + e_{257} + e_{29B} + e_{2CE} + e_{2DF} + e_{347} + e_{356} + e_{38B} + e_{3CF} + e_{3DE} + e_{45A} + e_{48C} + e_{49D} - e_{4BF} + e_{58D} + e_{59C} + e_{5BE} + e_{67A} + e_{68E} + e_{69F} + e_{6BD} + e_{78F} + e_{79E} + e_{7BC} + e_{89A} + e_{ACD} + e_{AEF}$
$\mathbb{P}_{168}$	$e_{123} + e_{14B} + e_{15A} + e_{167} + e_{189} + e_{1CD} + e_{1EF} + e_{246} + e_{259} + e_{27B} + e_{28A} + e_{2CE} + e_{2DF} + e_{347} + e_{358} + e_{36B} + e_{39A} + e_{3CF} + e_{3DE} + e_{45F} + e_{48C} + e_{49D} - e_{4AE} + e_{56D} + e_{57C} + e_{5BE} + e_{68E} + e_{69F} + e_{6AC} + e_{78F} + e_{79E} + e_{7AD} + e_{8BD} + e_{9BC} + e_{ABF}$
$\mathbb{P}_{170}$	$e_{123} + e_{14A} + e_{15B} + e_{167} + e_{189} + e_{1CD} + e_{1EF} + e_{246} + e_{258} + e_{27A} + e_{29B} + e_{2CE} - e_{2DF} + e_{347} + e_{359} + e_{36A} + e_{38B} + e_{3CF} + e_{3DE} + e_{45E} + e_{48C} + e_{49D} + e_{4BF} + e_{56C} + e_{57D} + e_{5AF} + e_{68E} + e_{69F} + e_{6BD} + e_{78F} + e_{79E} + e_{7BC} + e_{8AD} + e_{9AC} + e_{ABE}$

Table 6: Sedenion Quasi-Calibrations (continued)

P ₂₂₀	e ₁₂₃ +e ₁₄₅ +e ₁₆₇ +e ₁₈₉ +e _{1AB} +e _{1CD} +e _{1EF} +e ₂₄₆ +e ₂₅₇ +e _{28A} +e _{29B} +e _{2CE} +e _{2DF} +e ₃₄₇ +e ₃₅₆ +e _{38B} +e _{39A} +e _{3CF} +e _{3DE} +e _{48C} +e _{49D} +e _{4AE} +e _{4BF} +e _{58D} +e _{59C} +e _{5AF} +e _{5BE} -e _{68E} +e _{69F} +e _{6AC} +e _{6BD} +e _{78F} +e _{79E} +e _{7AD} +e _{7BC}
P ₂₂₂	e ₁₂₃ +e ₁₄₅ +e ₁₆₇ +e ₁₈₉ +e _{1AC} +e _{1BD} +e _{1EF} +e ₂₄₆ +e ₂₅₇ +e _{28A} +e _{29C} +e _{2BE} +e _{2DF} +e ₃₄₇ +e ₃₅₆ +e _{38C} +e _{39A} -e _{3BF} +e _{3DE} +e _{48B} +e _{49D} +e _{4AE} +e _{4CF} +e _{58D} +e _{59B} +e _{5AF} +e _{5CE} +e _{68E} +e _{69F} +e _{6AB} +e _{6CD} +e _{78F} +e _{79E} +e _{7AD} +e _{7BC}
P ₂₂₄	e ₁₂₃ +e ₁₄₅ +e ₁₆₇ +e ₁₈₉ +e _{1AB} +e _{1CD} +e _{1EF} -e ₂₄₆ +e ₂₅₇ +e _{28A} +e _{29B} +e _{2CE} +e _{2DF} +e ₃₄₇ +e ₃₅₆ +e _{38B} +e _{39A} +e _{3CF} +e _{3DE} +e _{48C} +e _{49D} +e _{4AE} +e _{4BF} +e _{58D} +e _{59C} +e _{5AF} +e _{5BE} +e _{68E} +e _{69F} +e _{6AC} +e _{6BD} +e _{78F} +e _{79E} +e _{7AD} +e _{7BC}
P ₂₂₆	e ₁₂₃ +e ₁₄₅ +e ₁₆₈ +e ₁₇₉ +e _{1AB} +e _{1CD} +e _{1EF} +e ₂₄₉ +e ₂₅₇ -e _{26B} +e _{28A} +e _{2CE} +e _{2DF} +e ₃₄₇ +e ₃₅₉ +e _{36A} +e _{38B} +e _{3CF} +e _{3DE} +e _{46D} +e _{48C} +e _{4AE} +e _{4BF} +e _{56C} +e _{58D} +e _{5AF} +e _{5BE} +e _{67E} +e _{69F} +e _{78F} +e _{7AD} +e _{7BC} +e _{89E} +e _{9AC} +e _{9BD}
P ₂₂₈	e ₁₂₃ +e ₁₄₅ +e ₁₆₇ +e ₁₈₉ +e _{1AB} +e _{1CD} +e _{1EF} +e ₂₄₆ +e ₂₅₇ +e _{28A} +e _{29B} +e _{2CE} +e _{2DF} +e ₃₄₇ +e ₃₅₆ +e _{38B} +e _{39A} +e _{3CF} +e _{3DE} +e _{48C} -e _{49D} +e _{4AE} +e _{4BF} +e _{58D} +e _{59C} +e _{5AF} +e _{5BE} +e _{68E} +e _{69F} +e _{6AC} +e _{6BD} +e _{78F} +e _{79E} +e _{7AD} +e _{7BC}
P ₂₃₀	e ₁₂₃ +e ₁₄₅ +e ₁₆₈ +e ₁₇₉ +e _{1AB} +e _{1CD} +e _{1EF} +e ₂₄₆ +e ₂₅₈ +e _{27A} +e _{29B} +e _{2CE} +e _{2DF} +e ₃₄₈ +e ₃₅₆ +e _{37B} +e _{39A} +e _{3CF} +e _{3DE} +e _{47C} +e _{49D} +e _{4AE} +e _{4BF} +e _{57D} -e _{59C} +e _{5AF} +e _{5BE} +e _{67E} +e _{69F} +e _{6AC} +e _{6BD} +e _{78F} +e _{89E} +e _{8AD} +e _{8BC}
P ₂₃₂	e ₁₂₃ +e ₁₄₅ +e _{16D} +e _{17C} +e ₁₈₉ +e _{1AB} +e _{1EF} +e _{24C} +e ₂₅₇ +e _{26E} +e _{28A} +e _{29B} +e _{2DF} +e ₃₄₇ +e _{35C} +e _{36F} +e _{38B} +e _{39A} +e _{3DE} +e ₄₆₈ +e _{49D} +e _{4AE} +e _{4BF} +e ₅₆₉ +e _{58D} +e _{5AF} +e _{5BE} +e _{67B} +e _{6AC} +e _{78F} +e _{79E} -e _{7AD} +e _{8CE} +e _{9CF} +e _{BCD}
P ₂₃₄	e ₁₂₃ +e ₁₄₅ +e ₁₆₈ +e ₁₇₉ +e _{1AB} +e _{1CD} +e _{1EF} +e ₂₄₆ +e ₂₅₈ -e _{27A} +e _{29B} +e _{2CE} +e _{2DF} +e ₃₄₈ +e ₃₅₆ +e _{37B} +e _{39A} +e _{3CF} +e _{3DE} +e _{47C} +e _{49D} +e _{4AE} +e _{4BF} +e _{57D} +e _{59C} +e _{5AF} +e _{5BE} +e _{67E} +e _{69F} +e _{6AC} +e _{6BD} +e _{78F} +e _{89E} +e _{8AD} +e _{8BC}
P ₂₃₆	e ₁₂₃ +e ₁₄₅ +e ₁₆₇ +e ₁₈₉ +e _{1AB} +e _{1CD} +e _{1EF} +e ₂₄₆ +e ₂₅₇ +e _{28A} +e _{29B} +e _{2CE} +e _{2DF} +e ₃₄₇ +e ₃₅₆ +e _{38B} +e _{39A} +e _{3CF} +e _{3DE} +e _{48C} +e _{49D} +e _{4AE} +e _{4BF} -e _{58D} +e _{59C} +e _{5AF} +e _{5BE} +e _{68E} +e _{69F} +e _{6AC} +e _{6BD} +e _{78F} +e _{79E} +e _{7AD} +e _{7BC}
P ₂₃₈	e ₁₂₃ +e ₁₄₅ +e ₁₆₇ +e ₁₈₉ +e _{1AC} +e _{1BD} +e _{1EF} +e ₂₄₆ +e ₂₅₇ +e _{28A} +e _{29C} -e _{2BE} +e _{2DF} +e ₃₄₇ +e ₃₅₆ +e _{38C} +e _{39A} +e _{3BF} +e _{3DE} +e _{48B} +e _{49D} +e _{4AE} +e _{4CF} +e _{58D} +e _{59B} +e _{5AF} +e _{5CE} +e _{68E} +e _{69F} +e _{6AB} +e _{6CD} +e _{78F} +e _{79E} +e _{7AD} +e _{7BC}
P ₂₄₀	e ₁₂₃ +e ₁₄₅ +e _{16C} +e _{17D} +e ₁₈₉ +e _{1AB} +e _{1EF} +e ₂₄₆ +e _{25C} +e _{27E} +e _{28A} +e _{29B} +e _{2DF} +e _{34C} +e ₃₅₆ +e _{37F} +e _{38B} +e _{39A} +e _{3DE} +e ₄₇₈ +e _{49D} +e _{4AE} +e _{4BF} +e ₅₇₉ +e _{58D} +e _{5AF} +e _{5BE} -e _{67A} +e _{68E} +e _{69F} +e _{6BD} +e _{7BC} +e _{8CF} +e _{9CE} +e _{ACD}
P ₂₄₂	e ₁₂₃ +e ₁₄₅ +e _{16D} +e _{17C} +e ₁₈₉ +e _{1AB} +e _{1EF} +e

Table 7: Sedenion Quasi-Calibrations (continued)

$\mathbb{P}_{268}$	$e_{123} + e_{145} + e_{167} + e_{189} + e_{1AC} + e_{1BD} + e_{1EF} + e_{24D} + e_{25B} - e_{268} + e_{279} + e_{2AF} + e_{2CE} + e_{34B} + e_{35D} + e_{369} + e_{378} + e_{3AE} + e_{3CF} + e_{46E} + e_{47F} + e_{48C} + e_{49A} + e_{56F} + e_{57E} + e_{58A} + e_{59C} + e_{6AB} + e_{6CD} + e_{7AD} + e_{7BC} + e_{8BF} + e_{8DE} + e_{9BE} + e_{9DF}$
$\mathbb{P}_{270}$	$e_{123} + e_{145} + e_{167} + e_{189} + e_{1AC} + e_{1BD} + e_{1EF} + e_{24B} + e_{25D} + e_{269} - e_{278} + e_{2AF} + e_{2CE} + e_{34D} - e_{35B} + e_{368} + e_{379} + e_{3AE} + e_{3CF} + e_{46F} + e_{47E} + e_{48C} + e_{49A} + e_{56E} + e_{57F} + e_{58A} + e_{59C} + e_{6AB} + e_{6CD} + e_{7AD} + e_{7BC} + e_{8BE} + e_{8DF} + e_{9BF} + e_{9DE}$
$\mathbb{P}_{272}$	$e_{123} + e_{145} + e_{167} + e_{189} + e_{1AC} + e_{1BD} + e_{1EF} + e_{24B} + e_{25D} + e_{269} - e_{278} + e_{2AF} + e_{2CE} + e_{34D} + e_{35B} + e_{368} + e_{379} + e_{3AE} + e_{3CF} + e_{46F} + e_{47E} + e_{48C} + e_{49A} + e_{56E} + e_{57F} + e_{58A} + e_{59C} + e_{6AB} + e_{6CD} + e_{7AD} + e_{7BC} + e_{8BE} + e_{8DF} + e_{9BF} + e_{9DE}$
$\mathbb{P}_{274}$	$e_{123} + e_{14D} + e_{15C} + e_{167} + e_{189} + e_{1AB} + e_{1EF} + e_{246} + e_{25F} + e_{27D} - e_{28A} + e_{29B} + e_{2CE} + e_{347} + e_{35E} + e_{36D} + e_{38B} + e_{39A} + e_{3CF} + e_{459} + e_{48C} + e_{4AE} + e_{4BF} + e_{56B} + e_{57A} + e_{58D} + e_{68E} + e_{69F} + e_{6AC} + e_{78F} + e_{79E} + e_{7BC} + e_{9CD} + e_{ADF} + e_{BDE}$
$\mathbb{P}_{276}$	$e_{123} + e_{145} + e_{167} + e_{189} + e_{1AC} + e_{1BD} + e_{1EF} + e_{24D} + e_{25B} + e_{268} + e_{279} + e_{2AF} + e_{2CE} + e_{34B} + e_{35D} + e_{369} + e_{378} + e_{3AE} + e_{3CF} - e_{46E} + e_{47F} + e_{48C} + e_{49A} + e_{56F} + e_{57E} + e_{58A} + e_{59C} + e_{6AB} + e_{6CD} + e_{7AD} + e_{7BC} + e_{8BF} + e_{8DE} + e_{9BE} + e_{9DF}$
$\mathbb{P}_{278}$	$e_{123} + e_{145} + e_{167} + e_{189} + e_{1AC} + e_{1BD} + e_{1EF} + e_{24B} + e_{25D} + e_{269} + e_{278} + e_{2AF} - e_{2CE} + e_{34D} + e_{35B} + e_{368} + e_{379} + e_{3AE} + e_{3CF} + e_{46F} + e_{47E} + e_{48C} + e_{49A} + e_{56E} + e_{57F} + e_{58A} + e_{59C} + e_{6AB} + e_{6CD} + e_{7AD} + e_{7BC} + e_{8BE} + e_{8DF} + e_{9BF} + e_{9DE}$
$\mathbb{P}_{280}$	$e_{123} + e_{145} + e_{167} + e_{189} - e_{1AC} + e_{1BD} + e_{1EF} + e_{24B} + e_{25D} + e_{269} + e_{278} + e_{2AF} + e_{2CE} + e_{34D} + e_{35B} + e_{368} + e_{379} + e_{3AE} + e_{3CF} + e_{46F} + e_{47E} + e_{48C} + e_{49A} + e_{56E} + e_{57F} + e_{58A} + e_{59C} + e_{6AB} + e_{6CD} + e_{7AD} + e_{7BC} + e_{8BE} + e_{8DF} + e_{9BF} + e_{9DE}$
$\mathbb{P}_{282}$	$e_{123} + e_{145} + e_{167} + e_{189} + e_{1AC} + e_{1BD} + e_{1EF} + e_{24B} + e_{25D} + e_{269} + e_{278} + e_{2AF} + e_{2CE} + e_{34D} + e_{35B} + e_{368} + e_{379} + e_{3AE} + e_{3CF} + e_{46F} + e_{47E} + e_{48C} + e_{49A} + e_{56E} + e_{57F} + e_{58A} + e_{59C} + e_{6AB} + e_{6CD} + e_{7AD} + e_{7BC} - e_{8BE} + e_{8DF} + e_{9BF} + e_{9DE}$
$\mathbb{P}_{284}$	$e_{123} + e_{145} + e_{168} + e_{17B} + e_{19F} + e_{1AE} + e_{1CD} + e_{24B} + e_{257} + e_{269} + e_{28F} + e_{2AD} + e_{2CE} + e_{347} + e_{35B} + e_{36F} + e_{389} + e_{3AC} + e_{3DE} + e_{46D} + e_{48C} + e_{49A} + e_{4EF} + e_{56C} + e_{58D} + e_{59E} - e_{5AF} + e_{67E} + e_{6AB} + e_{78A} + e_{79C} + e_{7DF} + e_{8BE} + e_{9BD} + e_{BCF}$
$\mathbb{P}_{286}$	$e_{123} + e_{145} + e_{167} + e_{189} + e_{1AC} + e_{1BD} + e_{1EF} - e_{24B} + e_{25D} + e_{269} + e_{278} + e_{2AF} + e_{2CE} + e_{34D} + e_{35B} + e_{368} + e_{379} + e_{3AE} + e_{3CF} + e_{46F} + e_{47E} + e_{48C} + e_{49A} + e_{56E} + e_{57F} + e_{58A} + e_{59C} + e_{6AB} + e_{6CD} + e_{7AD} + e_{7BC} + e_{8BE} + e_{8DF} + e_{9BF} + e_{9DE}$
$\mathbb{P}_{288}$	$e_{123} + e_{145} + e_{167} + e_{189} + e_{1AC} + e_{1BD} + e_{1EF} + e_{24B} + e_{25D} + e_{269} + e_{278} + e_{2AF} + e_{2CE} + e_{34D} + e_{35B} + e_{368} - e_{379} + e_{3AE} + e_{3CF} + e_{46F} + e_{47E} + e_{48C} + e_{49A} + e_{56E} + e_{57F} + e_{58A} + e_{59C} + e_{6AB} + e_{6CD} + e_{7AD} + e_{7BC} + e_{8BE} + e_{8DF} + e_{9BF} + e_{9DE}$
$\mathbb{P}_{290}$	$e_{123} + e_{14D} + e_{15C} + e_{167} + e_{189} + e_{1AB} + e_{1EF} + e_{246} + e_{25F} + e_{27D} + e_{28A} + e_{29B} + e_{2CE} + e_{347} + e_{35E} + e_{36D} + e_{38B} + e_{39A} + e_{3CF} + e_{459} + e_{48C} + e_{4AE} + e_{4BF} + e_{56B} + e_{57A} - e_{58D} + e_{68E} + e_{69F} + e_{6AC} + e_{78F} + e_{79E} + e_{7BC} + e_{9CD} + e_{ADF} + e_{BDE}$
$\mathbb{P}_{292}$	$e_{123} + e_{145} + e_{168} + e_{17E} + e_{19A} + e_{1BF} + e_{1CD} + e_{249} + e_{25A} - e_{26B} + e_{27D} + e_{28F} + e_{2CE} + e_{34A} + e_{359} + e_{36F} + e_{37C} + e_{38B} + e_{3DE} + e_{46D} + e_{47B} + e_{48C} + e_{4EF} + e_{56C} + e_{57F} + e_{58D} + e_{5BE} + e_{679} + e_{6AE} + e_{78A} + e_{89E} + e_{9BD} + e_{9CF} + e_{ABC} + e_{ADF}$
$\mathbb{P}_{294}$	$e_{123} + e_{145} + e_{167} + e_{189} + e_{1AC} + e_{1BD} + e_{1EF} + e_{24B} + e_{25D} + e_{269} + e_{278} + e_{2AF} + e_{2CE} + e_{34D} + e_{35B} + e_{368} + e_{379} + e_{3AE} - e_{3CF} + e_{46F} + e_{47E} + e_{48C} + e_{49A} + e_{56E} + e_{57F} + e_{58A} + e_{59C} + e_{6AB} + e_{6CD} + e_{7AD} + e_{7BC} + e_{8BE} + e_{8DF} + e_{9BF} + e_{9DE}$
$\mathbb{P}_{296}$	$e_{123} + e_{145} + e_{16F} + e_{17D} + e_{18A} + e_{19B} + e_{1CE} + e_{24D} + e_{257} + e_{26C} + e_{289} + e_{2AB} + e_{2EF} + e_{347} + e_{35D} + e_{36E} + e_{38B} + e_{39A} + e_{3CF} + e_{469} - e_{48C} + e_{4AE} + e_{4BF} + e_{56B} + e_{58E} + e_{59F} + e_{5AC} + e_{67A} + e_{68D} + e_{78F} + e_{79E} + e_{7BC} + e_{9CD} + e_{ADF} + e_{BDE}$
$\mathbb{P}_{298}$	$e_{123} + e_{146} + e_{15C} + e_{17A} + e_{189} + e_{1BD} + e_{1EF} + e_{245} + e_{26C} + e_{279} + e_{28A} + e_{2BE} + e_{2DF} + e_{34C} + e_{356} + e_{378} + e_{39A} + e_{3BF} + e_{3DE} + e_{47F} + e_{48B} + e_{49D} + e_{4AE} + e_{57D} + e_{58E} + e_{59F} + e_{5AB} + e_{67E} + e_{68D} + e_{69B} - e_{6AF} + e_{7BC} + e_{8CF} + e_{9CE} + e_{ACD}$
$\mathbb{P}_{300}$	$e_{123} + e_{146} + e_{15E} + e_{17D} + e_{18A} + e_{19B} + e_{1CF} + e_{24D} + e_{25C} + e_{267} + e_{289} + e_{2AB} + e_{2EF} + e_{347} + e_{35F} + e_{36D} + e_{38B} + e_{39A} + e_{3CE} + e_{459} - e_{48C} + e_{4AF} + e_{4BE} + e_{56B} + e_{57A} + e_{58D} + e_{68F} + e_{69E} + e_{6AC} + e_{78E} + e_{79F} + e_{7BC} + e_{9CD} + e_{ADE} + e_{BDF}$
$\mathbb{P}_{302}$	$e_{123} + e_{145} + e_{16F} + e_{17D} + e_{189} + e_{1AB} + e_{1CE} + e_{24D} + e_{257} + e_{26C} + e_{28B} + e_{29A} + e_{2EF} + e_{347} + e_{35D} + e_{36E} + e_{38A} + e_{39B} + e_{3CF} + e_{468} + e_{49F} + e_{4AE} + e_{4BC} + e_{569} + e_{58F} + e_{5AC} + e_{5BE} + e_{67A} + e_{6BD} + e_{78E} + e_{79C} - e_{7BF} + e_{8CD} + e_{9DE} + e_{ADF}$
$\mathbb{P}_{304}$	$e_{123} + e_{14D} + e_{15C} + e_{167} + e_{189} + e_{1AB} + e_{1EF} + e_{246} + e_{25F} + e_{27D} + e_{28A} + e_{29B} + e_{2CE} + e_{347} + e_{35E} + e_{36D} + e_{38B} + e_{39A} + e_{3CF} + e_{459} - e_{48C} + e_{4AE} + e_{4BF} + e_{56B} + e_{57A} + e_{58D} + e_{68E} + e_{69F} + e_{6AC} + e_{78F} + e_{79E} + e_{7BC} + e_{9CD} + e_{ADF} + e_{BDE}$
$\mathbb{P}_{306}$	$e_{123} + e_{145} + e_{16D} + e_{178} + e_{19A} + e_{1BC} + e_{1EF} + e_{248} + e_{257} + e_{26E} + e_{29B} + e_{2AC} + e_{2DF} + e_{347} + e_{358} + e_{36F} + e_{39C} + e_{3AB} + e_{3DE} + e_{46A} - e_{49D} + e_{4BF} + e_{4CE} + e_{569} + e_{5AD} + e_{5BE} + e_{5CF} + e_{67B} + e_{68C} + e_{79E} + e_{7AF} + e_{7CD} + e_{89F} + e_{8AE} + e_{8BD}$
$\mathbb{P}_{308}$	$e_{123} + e_{145} + e_{16A} + e_{178} + e_{19C} + e_{1BD} + e_{1EF} + e_{24B} + e_{25D} + e_{267} + e_{28A} + e_{29F} + e_{2CE} + e_{34D} + e_{35B} + e_{368} + e_{37A} + e_{39E} + e_{3CF} + e_{46F} + e_{479} + e_{48C} + e_{4AE} + e_{56E} + e_{57C} + e_{589} - e_{5AF} + e_{69B} + e_{6CD} + e_{7BF} + e_{7DE} + e_{8BE} + e_{8DF} + e_{9AD} + e_{ABC}$
$\mathbb{P}_{310}$	$e_{123} + e_{145} + e_{16D} + e_{17C} + e_{189} + e_{1AF} + e_{1BE} + e_{247} + e_{25C} + e_{26E} + e_{28A} + e_{29F} + e_{2BD} + e_{34C} + e_{357} + e_{36B} + e_{38F} + e_{39A} + e_{3DE} + e_{468} + e_{49D} + e_{4AE} - e_{4BF} + e_{569} + e_{58D} + e_{5AB} + e_{5EF} + e_{67A} + e_{6CF} + e_{78E} + e_{79B} + e_{7DF} + e_{8BC} + e_{9CE} + e_{ACD}$
$\mathbb{P}_{312}$	$e_{12E} + e_{13D} + e_{148} + e_{159} + e_{16C} + e_{17B} + e_{1AF} + e_{23C} + e_{24F} + e_{257} + e_{26D} + e_{28A} + e_{29B} + e_{347} + e_{35F} + e_{36E} + e_{38B} + e_{39A} + e_{45C} + e_{469} + e_{4AE} + e_{4BD} + e_{568} + e_{5AD} + e_{5BE} + e_{67A} - e_{6BF} + e_{78D} + e_{79E} + e_{7CF} + e_{89C} + e_{8EF} + e_{9DF} + e_{ABC} + e_{CDE}$
$\mathbb{P}_{314}$	$e_{125} + e_{13C} + e_{14D} + e_{169} + e_{178} + e_{1AF} + e_{1BE} + e_{237} + e_{246} + e_{28C} + e_{29D} - e_{2AE} + e_{2BF} + e_{34B} + e_{358} + e_{36F} + e_{39A} + e_{3DE} + e_{459} + e_{47F} + e_{48A} + e_{4CE} + e_{56D} + e_{57C} + e_{5AB} + e_{5EF} + e_{67B} + e_{68E} + e_{6AC} + e_{79E} + e_{7AD} + e_{89B} + e_{8DF} + e_{9CF} + e_{BCD}$

Table 8: Sedenion Quasi-Calibrations (continued)

$\mathbb{P}_{316}$	$e_{12A} + e_{137} + e_{14E} + e_{156} + e_{18C} + e_{19D} - e_{1BF} + e_{23D} + e_{24F} + e_{25C} + e_{268} + e_{279}$ $+ e_{2BE} + e_{345} + e_{36E} + e_{38B} + e_{39A} + e_{3CF} + e_{467} + e_{489} + e_{4AB} + e_{4CD} + e_{57E} + e_{58A}$ $+ e_{59B} + e_{5DF} + e_{69F} + e_{6AC} + e_{6BD} + e_{78F} + e_{7AD} + e_{7BC} + e_{8DE} + e_{9CE} + e_{AEF}$
$\mathbb{P}_{318}$	$e_{124} + e_{135} + e_{167} + e_{189} + e_{1AB} + e_{1CD} + e_{1EF} + e_{23D} + e_{25C} + e_{268} + e_{279} + e_{2AF}$ $+ e_{2BE} - e_{34C} + e_{36E} + e_{37F} + e_{38B} + e_{39A} + e_{45D} + e_{469} + e_{478} + e_{4AE} + e_{4BF} + e_{56F}$ $+ e_{57E} + e_{58A} + e_{59B} + e_{6AC} + e_{6BD} + e_{7AD} + e_{7BC} + e_{8CF} + e_{8DE} + e_{9CE} + e_{9DF}$
$\mathbb{P}_{320}$	$e_{123} + e_{145} + e_{167} + e_{189} + e_{1AC} + e_{1BD} + e_{1EF} + e_{24B} + e_{25D} + e_{269} + e_{278} + e_{2AF}$ $+ e_{2CE} + e_{34D} + e_{35B} + e_{368} + e_{379} + e_{3AE} + e_{3CF} + e_{46F} + e_{47E} + e_{48C} + e_{49A} + e_{56E}$ $+ e_{57F} + e_{58A} + e_{59C} + e_{6AB} + e_{6CD} + e_{7AD} - e_{7BC} + e_{8BE} + e_{8DF} + e_{9BF} + e_{9DE}$
$\mathbb{P}_{322}$	$e_{124} + e_{135} + e_{167} + e_{189} + e_{1AB} + e_{1CD} + e_{1EF} + e_{23C} + e_{25D} + e_{269} + e_{278} + e_{2AF}$ $+ e_{2BE} - e_{34D} + e_{36F} + e_{37E} + e_{38B} + e_{39A} + e_{45C} + e_{468} + e_{479} + e_{4AE} + e_{4BF} + e_{56E}$ $+ e_{57F} + e_{58A} + e_{59B} + e_{6AC} + e_{6BD} + e_{7AD} + e_{7BC} + e_{8CE} + e_{8DF} + e_{9CF} + e_{9DE}$
$\mathbb{P}_{324}$	$e_{123} + e_{14B} + e_{159} + e_{167} + e_{18F} + e_{1AE} + e_{1CD} + e_{24D} + e_{256} + e_{279} + e_{28A} + e_{2BC}$ $+ e_{2EF} + e_{34C} + e_{357} + e_{369} + e_{38E} + e_{3AF} + e_{3BD} + e_{45E} + e_{46F} + e_{478} + e_{49A} + e_{58C}$ $+ e_{5AB} + e_{5DF} + e_{68B} + e_{6AC} + e_{6DE} - e_{7AD} + e_{7BF} + e_{7CE} + e_{89D} + e_{9BE} + e_{9CF}$
$\mathbb{P}_{326}$	$e_{126} + e_{137} + e_{14E} + e_{15F} + e_{18B} + e_{19C} - e_{1AD} + e_{234} + e_{258} + e_{27E} + e_{29A} + e_{2BF}$ $+ e_{2CD} + e_{359} + e_{36E} + e_{38A} + e_{3BD} + e_{3CF} + e_{45A} + e_{467} + e_{489} + e_{4BC} + e_{4DF} + e_{56B}$ $+ e_{57C} + e_{5DE} + e_{68F} + e_{69D} + e_{6AC} + e_{78D} + e_{79F} + e_{7AB} + e_{8CE} + e_{9BE} + e_{AEF}$
$\mathbb{P}_{328}$	$e_{12B} + e_{13A} + e_{147} + e_{156} + e_{18D} + e_{19C} - e_{1EF} + e_{23F} + e_{24C} + e_{25D} + e_{268} + e_{279}$ $+ e_{2AE} + e_{34D} + e_{35C} + e_{369} + e_{378} + e_{3BE} + e_{45F} + e_{46E} + e_{48A} + e_{49B} + e_{57E} + e_{58B}$ $+ e_{59A} + e_{67F} + e_{6AC} + e_{6BD} + e_{7AD} + e_{7BC} + e_{89F} + e_{8CE} + e_{9DE} + e_{ABF} + e_{CDF}$